A Hopf Algebra on Proof Trees over a Nonmonotonic Consequence Relation

Abstract

We construct a pre-Lie algebra on proof trees for a multisuccedent sequent calculus equipped to a primitive base consequence relation which is not assumed to be monotonic. The sequent calculus naturally gives rise to decorated and labelled rooted trees, while the inadmissibility of standard Weakening rules gives rise to a constrained grafting operation on trees. We define a proof-theoretically meaningful equivalence relation which identifies proof-trees differing only in inessential or bureaucratic scheduling or reassociation choices. Passing to the induced quotient we show that there is a clean separation between cancelling and genuinely asymmetric contributions to the associator, yielding a pre-Lie identity for proof-tree composition. This extends to a Hopf algebra by the Oudom-Guin construction. This suggests that Hopf-algebraic methods on proof trees are a potential tool for structural proof theory, in the sense that the presence or absence of structural rules controls the existence and form of the algebraic structure.

1 Introduction

The aim of this paper is to describe a proof-theoretically meaningful algebraic setting in which monotonicity of consequence and the familiar structural rules are not built in from the outset. This allows structural rules to be analysed as constraints on algebraic structures.

Our proof calculus is the Non-Monotonic Multi-Succedent sequent calculus NMMS introduced by Kaplan (2018). Nonmonotonicity of consequence here is represented proof-theoretically by the failure of Weakening:

Include Weakening rules here

Structural rules such as Weakening have of course been studied extensively in the context of substructural logics. Linear logic in particular controls extensions of contexts and multiplicity of hypotheses via constraints

on the Weakening and Contraction. Linear logic also possesses the Cut Elimination property which is most desirable from the point of view of computational content. NMMS is a very different setting – here the sequent calculus is equipped on a primitive base consequence relation $\mid\!\sim_0$, which serves to encode non-logical patterns of defeasible *material inference* of the sort studied by Brandom and collaborators in the *inferentialist* analysis of meaning in natural language. Monmonotonic consequence relations of this sort fail in general to admit the Cut rule:

The inadmissibility of Cut entails that NMMS does not admit a *categorical semantics* such that Cut is represented by composition of morphisms of some suitable category, as is the case for linear logic (*-autonomous or symmetric monoidal closed categories), intuitionistic logic (cartesian closed categories), and classical logic (control categories). Here we show that NMMS admits a non-trivial interpretation in a different algebraic structure, specifically a Hopf algebra defined over a quotient set of labelled and decorated rooted trees which are just the proof trees of the sequent calculus.

Hopf algebras on rooted trees are well-studied objects that have seen applications in areas ranging from renormalization for perturbative Quantum Field Theory (Connes and Kreimer) to syntactic trees in generative linguistics (Chomsky, Marcolli, Berwick). Hopf algebras for our specific setting of typed and decorated binary trees also have been studied in the combinatorics literature, especially by Foissy. To the author's knowledge no Hopf algebra for proof trees has yet been discussed explicitly. However the of the construction developed in this paper is

The nonmonotonicity property that makes NMMS unamenable to categorical semantics is in fact what makes the Hopf algebra construction nontrivial. The sequent calculus rules naturally admit an interpretation as a *grafting* operation on proof trees – but the absence of Weakening means that the grafting operation is *not* free to arbitrarily extend (or indeed to contract) contexts by structural rules. This makes the combinatorics of proof tree grafting more constrained and more intricate than the Weakening-admissible setting. In order to define a pre-Lie product directly from proof-tree grafting, we define a proof-theoretically meaningful equivalence relation on iterated graft compositions such that collections of proof trees that are pairwise distinguished only by artificial serialisation of logically concurrent inferences belong to the same equivalence class. Two graftings acting at non-interacting leaf positions may be performed in either order; the sequent calculus records this choice as part of the derivation witness, but neither

ordering carries proof-theoretic priority. This is a similar sort of "bureaucratic" redundancy to that which Girard removed in passing from sequent calculus to proof nets, and that Dosen placed at the centre of his analysis of identity of proofs. Without the quotient a naive associator comparison fails — the noise sector contributes asymmetrically across the two bracketings — and the pre-Lie identity cannot be established on raw proof trees, so obstructing the Hopf algebra construction.

The key point is that it is proof-theoretically meaningful quotient itself that determines the relevant rooted trees, the grafting operation, the quotient of bureaucratic symmetries, and ultimately the class-level associator comparison from which the pre-Lie identity emerges.

Two distinct issues then arise. The first is combinatorial: how do iterated graftings decompose? The second is proof-theoretic: which apparent distinctions between ordered pairs of grafts are genuine and which are merely bureaucratic presentation artefacts? The central contribution of the paper is to show that these two questions can be separated cleanly. The geometry of a graft pair yields a decomposition into independent and nested cases. Passing to a quotient of raw graft-pair data isolates the quotient-trivial overlap sector, leaves a rigid residual sector, and makes the associator comparison transparent.

The argument proceeds in three stages. First, one shows that every graft pair is governed by a simple rooted-tree geometry: the two insertions are either nested or independent. Second, one passes from raw graft-pair witnesses to a quotient that identifies the genuinely bureaucratic symmetries of proof composition. This separates a quotient-trivial noise sector from a residual sector. Third, one proves that the mixed residual case is eliminated, so that the residual contribution classes become rigid enough to be matched across the two bracketings of the associator.

Relation to substructural logics. It is worth distinguishing the nonmonotonicity studied here from the related but different phenomenon in linear logic. Linear logic exhibits what one might call *inner* nonmonotonicity: the inference connector $\multimap$ is not monotone, in the sense that provability of $\vdash A \multimap B$ does not imply provability of $\vdash X \otimes A \multimap B$. However, the *provability relation* of linear logic is itself monotone in the meta-sense: adding new axioms or rules cannot destroy existing proofs. The present setting is different: the primitive relation $\mid\!\sim_0$ is allowed to be nonmonotone in the *meta* sense, so that provability itself may be defeated by enlarging the premiss set. This is the stronger, logical notion of nonmonotonicity studied in the literature on defeasible and nonmonotonic reasoning, and it is what

makes the absence of weakening proof-theoretically essential rather than merely convenient.

The structural rules programme. The present paper establishes the base case of a broader research programme. The sequent calculus $NMMS(\mid\!\sim_0)$ is currently formulated with multisets (so exchange is built in) and without weakening or contraction. A natural next question is: which structural rules obstruct the existence of a compatible product and coproduct, and hence of a Hopf algebra on proof trees? Adding weakening would make the sequent interfaces non-rigid, potentially destroying the exactness of the leaf-matching constraint on which the coproduct relies. Adding contraction would identify multisets differing in formula multiplicities, changing the quotient structure in ways that may break the noise/residual separation. This programme — using Hopf-algebraic compatibility as a diagnostic for structural rules — is left for subsequent work, but it provides the conceptual frame within which the present results should be read.

Lean formalization. The main results of this paper have been fully formalized in Lean 4 using Mathlib. The central theorem `classLevel_preLie_identity` has been machine-verified and depends only on the standard foundational axioms `propext`, `choice`, and `Quot.sound`. In particular, no `sorry` placeholders appear in the live proof chain. The formalization comprises approximately 19 000 lines across three main files and served as the primary vehicle through which the quotient structure and the noise/residual separation were discovered; the prose paper is a conservative account of what the formal development established.

The presentation is intentionally more conservative than the internal Lean development. We keep only the mathematically significant layers of the construction in the main text:

$$\begin{aligned}\text{primitive consequence} &\longrightarrow \text{derivability}\\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting}\\ &\longrightarrow \text{graft-pair quotient} \longrightarrow \text{class-level pre-Lie identity.}\end{aligned}$$

The paper is organized accordingly. Section 2 defines the primitive consequence relation, the calculus $NMMS(\mid\!\sim_0)$, and its induced derivability relation. Section 3 develops proof trees as rooted ordered objects. Section 4 then recalls the algebraic notion of pre-Lie algebra to which the later quotient analysis leads. Sections 5 and 6 develop grafting and the graft-pair decomposition underlying the associator analysis. Sections 7–9 contain the main quotient argument: bureaucratic equivalence, the separation of noise

from residual contribution classes, elimination of mixed residual classes, and the class-level pre-Lie identity. Section 10 records the cut-based coproduct material only as preparatory structure for future work.

2 Primitive consequence and induced derivability

2.1 Base vocabulary and full language

Let L_0 be a set of atomic sentence letters. We set

$$L_B := L_0 \cup \{\bot\},$$

where $\bot$ is treated as a primitive incompatibility marker.

Definition 1. *The full language L is generated inductively from the base fragment L_B by*

$$A ::= p \mid \bot \mid (A \Rightarrow A) \mid (A \wedge A) \mid (A \vee A) \mid \neg A, \qquad p \in L_0.$$

Remark 2. *At the base level, $\bot$ is not governed by any special elimination rule. It functions simply as a primitive marker of incompatibility. In particular, sequents such as $A, B, C \mathrel{|\!\!\sim_0} \bot$, with $A, B, C \in L_0$, may be used to record material incompatibility without imposing any general explosion principle.*

2.2 Primitive consequence

Definition 3. *A* primitive consequence relation *is a binary relation*

$$\mathrel{|\!\!\sim_0} \subseteq \mathit{MSet}(L_B) \times \mathit{MSet}(L_B),$$

where $\mathit{MSet}(L_B)$ denotes the finite multisets of formulas of the base fragment L_B.

Remark 4 (Order-style notation for primitive consequence)**.** *When it is convenient to emphasize the interface aspect of primitive consequence, we may write*

$$\Gamma \leq_0 \Theta \qquad \textit{for} \qquad \Gamma \mathrel{|\!\!\sim_0} \Theta.$$

This is only a change of notation: no reflexivity, transitivity, antisymmetry, or monotonicity is assumed. Thus $\leq_0$ is an order-like relation on base sequents, not necessarily a preorder. The point of this notation is simply to stress that the leaf labels of proof trees carry a primitive comparison relation which controls when one proof tree may be grafted into another.

Remark 5. *This is the proof-theoretically intended setup: primitive consequence is given only on the pre-logical fragment, and the logical vocabulary is added above it by the sequent rules.*

Remark 6. *No structural conditions are imposed on* $\mid\!\sim_0$. *In particular, it need not be monotone: from* $\Gamma \mid\!\sim_0 \Theta$ *one cannot in general infer either* $(\Gamma, A) \mid\!\sim_0 \Theta$ *or* $\Gamma \mid\!\sim_0 (\Theta, A)$.

2.3 Sequents

Definition 7. *The set of* multi-succedent sequents *over* L *is*

$$\mathit{Seq}(L) := \mathit{MSet}(L) \times \mathit{MSet}(L).$$

An element $(\Gamma, \Delta) \in \mathit{Seq}(L)$ *is written*

$$\Gamma \mid\!\sim \Delta,$$

where $\Gamma, \Delta \in \mathit{MSet}(L)$.

Remark 8. *The use of multisets makes multiplicity part of the syntax of a sequent. Accordingly, no structural principle is built into the sequent object beyond what is already forced by multiset equality.*

2.4 The calculus $\mathit{NMMS}(\mid\!\sim_0)$

We now define the nonmonotonic multi-succedent sequent calculus generated by $\mid\!\sim_0$.

Definition 9. *A* derivation *in* $\mathit{NMMS}(\mid\!\sim_0)$ *is an element of the least class closed under the following formation clauses.*

Base axiom. Whenever $\Gamma, \Theta \in \mathit{MSet}(L_B)$ and $\Gamma \mid\!\sim_0 \Theta$, we may form the sequent

$$\frac{\Gamma \mid\!\sim_0 \Theta}{\Gamma \mid\!\sim \Theta} \quad \textit{(Base)}.$$

Implication rules.

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \Rightarrow B), \Gamma \mid\!\sim \Theta} \quad (\Rightarrow L)$$

$$\frac{A, \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \Rightarrow B), \Theta} \quad (\Rightarrow R)$$

Conjunction rules.

$$\frac{A, B, \Gamma \mid\!\sim \Theta}{(A \wedge B), \Gamma \mid\!\sim \Theta} \quad (\wedge L)$$

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \wedge B), \Theta} \quad (\wedge R)$$

Disjunction rules.

$$\frac{A, \Gamma \mid\!\sim \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \vee B), \Gamma \mid\!\sim \Theta} \quad (\vee L)$$

$$\frac{\Gamma \mid\!\sim A, B, \Theta}{\Gamma \mid\!\sim (A \vee B), \Theta} \quad (\vee R)$$

Negation rules.

$$\frac{\Gamma \mid\!\sim A, \Theta}{\neg A, \Gamma \mid\!\sim \Theta} \quad (\neg L) \qquad \frac{A, \Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \neg A, \Theta} \quad (\neg R)$$

Remark 10. *The logical rules are standard active-formula rules. The important structural choice is not the shape of these rules but the absence of built-in structural principles such as weakening. This is the proof-theoretic setting in which the primitive relation $\mid\!\sim_0$ genuinely matters.*

Remark 11. *No weakening or contraction rules are assumed. In particular, the rules*

$$\frac{\Gamma \mid\!\sim \Theta}{\Gamma, A \mid\!\sim \Theta} \qquad \frac{\Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \Theta, A}$$

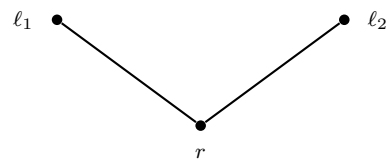
are not built into the system.

Remark 12. *The base relation is extended by logic rather than replaced by logic. In other words, NMMS($\mid\!\sim_0$) is the deductive closure generated from the primitive inferential relation $\mid\!\sim_0$.*

Remark 13. *The next two figures serve different purposes. The first displays a small closed derivation both proof-theoretically and graph-theoretically. The second is a schematic picture of grafting by substitution at a matching leaf.*

$$\dfrac{\dfrac{\Gamma \mid\!\sim_0 (A \Rightarrow B), \Theta}{\Gamma \mid\!\sim (A \Rightarrow B), \Theta}\,\textsc{Base} \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \Rightarrow B) \wedge C), \Theta}\,\wedge R$$

The same derivation may also be displayed in graph-theoretic form as an ordinary rooted ordered tree. To match the proof-theoretic orientation above, we place the root at the bottom and draw edges upward toward the premisses.



$$\begin{array}{rcll} \rho(r) & = & \wedge R, & \sigma(r) = \Gamma \mid\!\sim ((A \Rightarrow B) \wedge C), \Theta, \\ \rho(\ell_1) & = & \textsc{Base}, & \sigma(\ell_1) = \Gamma \mid\!\sim (A \Rightarrow B), \Theta, \\ \rho(\ell_2) & = & \textsc{Base}, & \sigma(\ell_2) = \Gamma \mid\!\sim C, \Theta. \end{array}$$

Formally, this is the same proof object viewed as a finite rooted ordered tree (V, E, r) together with a sequent-labelling map $\sigma : V \to \mathit{Seq}(L)$ and a rule-labelling map

$$\rho : V \to \{\mathsf{Base}, \Rightarrow L, \Rightarrow R, \wedge L, \wedge R, \vee L, \vee R, \neg L, \neg R\}.$$

If v is an internal vertex with ordered children $v_1, \ldots, v_n$, then $\sigma(v_1), \ldots, \sigma(v_n)$ are exactly the premisses of an instance of the rule $\rho(v)$ with conclusion $\sigma(v)$. If v is a leaf, then $\rho(v) = \mathsf{Base}$, and $\sigma(v)$ is justified by a primitive consequence instance.

2.5 Induced derivability

Definition 14. *For $\Gamma, \Delta \in \mathit{MSet}(L)$, we write*

$$\Gamma \vdash_{\mathit{NMMS}(\mid\!\sim_0)} \Delta$$

if there exists a derivation of the sequent $\Gamma \mid\!\sim \Delta$ in the calculus $\mathit{NMMS}(\mid\!\sim_0)$.

Remark 15. *This explicitly separates the primitive relation $\mid\!\sim_0$ from the induced derivability relation $\vdash_{\mathit{NMMS}(\mid\!\sim_0)}$. In a nonmonotonic setting, that distinction is mathematically significant.*

Remark 16 (Order-style notation for derivability)**.** *Likewise, one may write*

$$\Gamma \leq_{\mathit{NMMS}(\mid\!\sim_0)} \Delta \qquad \textit{for} \qquad \Gamma \vdash_{\mathit{NMMS}(\mid\!\sim_0)} \Delta.$$

In contrast with the primitive relation $\leq_0$, this is the deductive closure generated by the rules of the calculus. The passage

$$\leq_0 \longmapsto \leq_{NMMS(\mid\!\sim_0)}$$

is therefore the proof-theoretic closure step from primitive material consequence to full derivability.

3 Proof trees

3.1 Tree representation

Definition 17. *A* proof tree *is a tuple*

$$t = (V_t, E_t, r_t, \lambda_t, \sigma_t),$$

where (V_t, E_t, r_t) is a finite rooted ordered tree, $\sigma_t : V_t \to Seq(L)$ is a sequent-labelling, and

$$\lambda_t : V_t \to \{\mathsf{Base}, \Rightarrow L, \Rightarrow R, \wedge L, \wedge R, \vee L, \vee R, \neg L, \neg R\},$$

such that:

(i) *if v is a leaf, then $\lambda_t(v) = \mathsf{Base}$;*

(ii) *if v is an internal vertex, then $\lambda_t(v) \neq \mathsf{Base}$, and the ordered immediate subtrees of v are exactly the premise trees of an instance of the rule $\lambda_t(v)$ with conclusion equal to the sequent label of v.*

The sequent at the root is called the conclusion *of the proof tree.*

Remark 18. *The order on immediate subtrees is part of the formal object. This is the only reason addresses are later coded by finite sequences of child indices. Formally, if v is an internal vertex of a proof tree t, we may write*

$$\mathrm{ch}_t(v) = (v_0, \ldots, v_{k(v)-1})$$

for its ordered list of immediate successors. Conceptually, the objects are ordinary rooted proof trees.

Proposition 19. *Every derivation d in $NMMS(\mid\!\sim_0)$ determines a proof tree $Tree(d)$ with the same conclusion sequent.*

Proof. We argue by induction on the height of the derivation d.

If d has height 0, then d is a base derivation

$$\frac{\Gamma \mid\!\sim_0 \Theta}{\Gamma \mid\!\sim \Theta}.$$

We assign to d the one-vertex rooted tree whose unique vertex is labelled by the sequent $\Gamma \mid\!\sim \Theta$ and tagged by Base. Its root label is therefore the conclusion sequent of d.

Suppose now that d ends with an application of one of the logical rules of $NMMS(\mid\!\sim_0)$, with immediate subderivations $d_1, \ldots, d_n$. By the induction hypothesis, each d_i determines a finite rooted ordered proof tree $Tree(d_i)$ whose root carries the conclusion sequent of d_i. Form a new rooted ordered tree by adjoining a fresh root r, declaring the trees $Tree(d_1), \ldots, Tree(d_n)$ to be the ordered immediate subtrees of r, labelling r by the conclusion sequent of the last inference of d, and tagging r by the rule symbol used in that inference.

Because the premiss sequents of the last rule instance are exactly the conclusion sequents of the subderivations $d_1, \ldots, d_n$, the resulting rooted ordered tree satisfies the defining conditions of a proof tree. It is finite because it is obtained by adjoining one new root to finitely many finite trees. Its root sequent is, by construction, the conclusion sequent of d.

This completes the induction. □

Definition 20 (The set of proof trees)**.** *We write PTree for the set of all finite rooted ordered trees equipped with the rule labels and sequent labels induced by derivations of $NMMS(\mid\!\sim_0)$. Thus an element $t \in PTree$ may be written as*

$$t = (V_t, E_t, r_t, \lambda_t, \sigma_t),$$

where V_t is the finite vertex set, $E_t \subseteq V_t \times V_t$ is the edge relation directed away from the root r_t, λ_t records the rule label at each vertex, and σ_t records the sequent label. When the labels play no explicit role we suppress them and write simply $t = (V_t, E_t, r_t)$.

Remark 21. *All trees occurring below are elements of PTree. In particular, the raw grafting product and its linearization are defined on PTree itself; the quotient enters later at the level of* two-step witnesses *for the associator, not at the level of the underlying one-step product.*

3.2 Addresses

Definition 22. *An* address *in a proof tree is a finite sequence of natural numbers indicating a path from the root by successive child indices. The empty address selects the whole tree.*

Definition 23 (Address set and vertex map)**.** *If $t = (V_t, E_t, r_t)$ is a rooted ordered proof tree, we write*

$$Addr(t) \subseteq \mathbb{N}^{<\omega}$$

for the set of all finite sequences $a = (a_1, \ldots, a_n)$ for which there exists a chain of vertices

$$v_0 = r_t,\ v_1, \ldots, v_n \in V_t$$

such that, for each $i < n$, the vertex v_{i+1} is the a_{i+1}-th element of the ordered child list $\mathrm{ch}_t(v_i)$. The empty sequence $\langle\rangle$ belongs to $Addr(t)$ and names the root. We write

$$v_t : Addr(t) \to V_t$$

for the induced vertex map, sending an address to the corresponding vertex. If $a, c \in \mathbb{N}^{<\omega}$, we write $a{\cdot}c$ for concatenation of addresses. For addresses a, b, we write $a \preceq b$ if a is an initial segment of b, and $a \prec b$ if $a \preceq b$ and $a \neq b$. Thus b extends *a exactly when $a \preceq b$. Two addresses are* incomparable *if neither $a \preceq b$ nor $b \preceq a$.*

Remark 24. *Addresses are not conceptually basic. They are simply a convenient way of coding vertices and rooted subtrees in the formal development.*

Definition 25 (Address cones and rooted subtrees)**.** *Let $t \in PTree$ and $a \in Addr(t)$. The* address cone below *a is the principal upset*

$$Addr_t(a) := \{\, b \in Addr(t) \mid a \preceq b \,\}.$$

The rooted subtree of t at a *is denoted $t \downarrow_a$. Its address set is*

$$Addr(t \downarrow_a) := \{\, c \in \mathbb{N}^{<\omega} \mid a \cdot c \in Addr(t) \,\},$$

and its rule and sequent labels are inherited from those of t along the translation $c \mapsto a \cdot c$.

Proposition 26 (Disjointness of incomparable address cones)**.** *Let $t \in PTree$ and $a, b \in Addr(t)$. If a and b are incomparable, then*

$$Addr_t(a) \cap Addr_t(b) = \varnothing.$$

Proof. Suppose $d \in Addr_t(a) \cap Addr_t(b)$. Then $a \preceq d$ and $b \preceq d$. Since a and b are both initial segments of the same finite sequence d, one of them must be an initial segment of the other. Hence $a \preceq b$ or $b \preceq a$, contradicting incomparability. $\square$

Remark 27 (Finiteness of address chains)**.** *For each fixed proof tree t, the set $Addr(t)$ is finite, since the vertex set V_t is finite and the vertex map $v_t : Addr(t) \to V_t$ is bijection onto the vertices of t. Hence $(Addr(t), \preceq)$ is a finite partially ordered set. In particular, every chain*

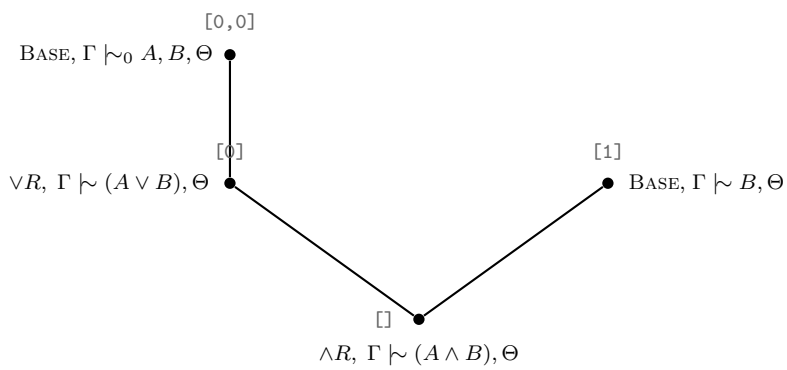
$$a_0 \prec a_1 \prec \cdots \prec a_k$$

of comparable addresses is finite. Thus for any fixed derivation, iterated nesting of graft sites may occur only finitely many times: one can move strictly downward in the tree only along a finite address chain.

Remark 28 (Use of cones in the two-step analysis)**.** *The notations $Addr_t(a)$ and $t\downarrow_a$ are used later in the graft-pair decomposition. If*

$$\mathrm{gr}_a(y, z) = z' \qquad \textit{and} \qquad \mathrm{gr}_b(x, z') = w,$$

then the inner *case is the condition $b \in Addr_{z'}(a)$, i.e. $a \preceq b$, while the* outer *case is the condition that a and b are incomparable in $(Addr(z'), \preceq)$. Proposition 25 is exactly the set-theoretic statement that the rooted regions affected by two outer grafts are disjoint.*



Each node carries a rule tag and a sequent label. Addresses (grey `monospace`) record paths from the root by child index: `[0]` is the first child of the root, `[1]` the second, and `[0,0]` the first child of `[0]`. The nodes at `[0,0]` and `[1]` are leaves (open premiss positions); the node at `[0]` is an internal node with one child.

4 Matching-leaf grafting

4.1 Local grafting

Definition 29. *Let u and t be proof trees. A leaf $a \in Addr(t)$ is* graftable for u *if the sequent labelling that leaf is exactly the conclusion of u.*

Definition 30. *A* matching-leaf grafting *of u into t is the proof tree obtained by replacing a graftable leaf of t by the tree u.*

Definition 31. *For fixed proof trees $u, t \in PTree$, we write $Graft(u, t)$ for the* finite family of one-step graft outputs *obtained by replacing a single graftable leaf of t by u. Equivalently, $Graft(u, t)$ is indexed by the set of graftable addresses in t.*

Definition 32 (Partial graft map)**.** *For proof trees u, t, let*

$$\mathrm{Leaf}_u(t) := \{\, a \in Addr(t) : a \text{ names a leaf of } t \text{ labelled by the conclusion sequent of } u \,\}.$$

The partial graft map

$$\gamma_{u,t} : \mathrm{Leaf}_u(t) \to PTree$$

is defined by $\gamma_{u,t}(a) = \mathrm{gr}_a(u, t)$, the proof tree obtained by grafting u into t at address a. We therefore regard $Graft(u, t)$ as the finite indexed family

$$Graft(u, t) = \big(\gamma_{u,t}(a)\big)_{a \in \mathrm{Leaf}_u(t)}.$$

Its set of distinct outputs is

$$\mathrm{SuppGraft}(u, t) := \mathrm{im}(\gamma_{u,t}),$$

and its multiplicity function is

$$m_{u,t}(w) := \#\{\, a \in \mathrm{Leaf}_u(t) \mid \gamma_{u,t}(a) = w \,\} \qquad (w \in PTree).$$

Definition 33 (Successful grafting)**.** *For fixed proof trees $u, t \in PTree$ and an address $a \in Addr(t)$, we say that the grafting of u into t at a is* successful *if $a \in \mathrm{Leaf}_u(t)$, equivalently if $\gamma_{u,t}(a)$ is defined.*

Proposition 34 (Finiteness of one-step graft outputs)**.** *For fixed $u, t \in PTree$, the indexing set $\mathrm{Leaf}_u(t)$ is finite. Consequently the family $Graft(u, t)$ of one-step graft outputs is finite.*

Proof. The set $\mathrm{Leaf}_u(t)$ is a subset of the finite address set $Addr(t)$. Hence it is finite. The family $Graft(u, t)$ is the image of $\gamma_{u,t}$ on this finite indexing set, so it is a finite indexed family. Its support set $\mathrm{SuppGraft}(u, t)$ is finite as the image of a finite set. □

Remark 35. *This operation is controlled by exact interface matching. It does not rely on extending contexts by structural rules.*

Remark 36. *For fixed $u, t \in PTree$,*

$$\mathrm{SuppGraft}(u,t) = \gamma_{u,t}\big(\mathrm{Leaf}_u(t)\big), \qquad |\,\mathrm{SuppGraft}(u,t)| \le |\,\mathrm{Leaf}_u(t)| \le |Addr(t)| < \infty.$$

Thus the one-step graft data is finite because it is indexed by the finite set $\mathrm{Leaf}_u(t)$*. It is not an iterated closure construction under the sequent rules.*

Matching-leaf grafting is substitution of one proof tree into a leaf of another carrying the same sequent interface. In the next display, the left tree t contains a base leaf labelled by $\Gamma \mid\!\sim (A \vee B), \Theta$. The middle tree u, shown in red, derives that same sequent from the stronger base sequent $\Gamma \mid\!\sim A, B, \Theta$ by an application of $(\vee R)$. Replacing the matching leaf of t by u yields the grafted derivation on the right.

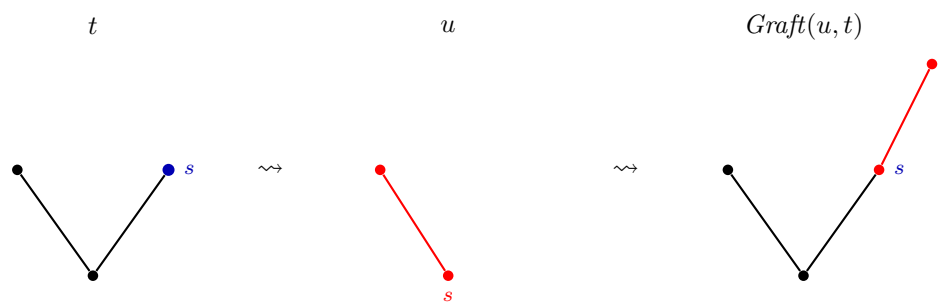
$$\dfrac{\dfrac{\Gamma \mid\!\sim_0 (A \vee B), \Theta}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\textsc{Base} \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \vee B) \wedge C), \Theta}\,\wedge R \qquad\qquad \dfrac{\dfrac{\Gamma \mid\!\sim_0 A, B, \Theta}{\Gamma \mid\!\sim A, B, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\vee R$$

$$t \qquad\qquad\qquad\qquad u$$

$$\dfrac{\dfrac{\dfrac{\Gamma \mid\!\sim_0 A, B, \Theta}{\Gamma \mid\!\sim A, B, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\vee R \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \vee B) \wedge C), \Theta}\,\wedge R$$

$$\mathit{Graft}(u,t)$$

The same grafting may also be shown graph-theoretically. The blue leaf of t and the root of u carry the same sequent s, while in $\mathit{Graft}(u,t)$ only the inserted top branch is shown in red.



Remark 37. *The Lean formalization is carried out in a slightly more general setting in which the primitive relation is allowed to be defined on multisets of formulas of the full language L. The intended proof-theoretic reading of the present paper is the restricted setup above, and all the formal results apply* a fortiori *to that case.*

4.2 Closure under derivability

Proposition 38 (Closure under derivability)**.** *If $u = \mathit{Tree}(d_{\text{in}})$ and $t = \mathit{Tree}(d_{\text{out}})$ arise from derivations and a is a graftable leaf of t for u, then the grafted tree is again the proof tree of a derivation in $\mathit{NMMS}(\mid\!\sim_0)$.*

Proof. We argue by induction on the height of d_{out}.

If d_{out} has height 0, then t consists of a single leaf. Since a is graftable for u, the conclusion sequent at that leaf is the conclusion of d_{in}. Therefore the grafted tree is exactly $u = \mathit{Tree}(d_{\text{in}})$, and hence is the proof tree of a derivation.

Now suppose d_{out} has positive height, and write its last inference as a rule instance

$$\rho(d_1, \ldots, d_k),$$

where $d_1, \ldots, d_k$ are the immediate subderivations and ρ is the final rule. Since a names a leaf of $t = \mathit{Tree}(d_{\text{out}})$, there is a unique index $i \in \{1, \ldots, k\}$ such that a lies in the address set of the i-th premise subtree $\mathit{Tree}(d_i)$; write the corresponding local address as $a_i \in \mathit{Addr}(\mathit{Tree}(d_i))$. By the inductive hypothesis, replacing the leaf of $\mathit{Tree}(d_i)$ at a_i by $u = \mathit{Tree}(d_{\text{in}})$ yields a proof tree $\mathit{Tree}(d_i')$ of some derivation d_i' having the same conclusion sequent as d_i, because graftability means that the replaced leaf and the root of u carry the same sequent label.

Now reapply the same final rule ρ to the tuple of premises

$$d_1, \ldots, d_{i-1}, d_i', d_{i+1}, \ldots, d_k.$$

The conclusions of the premises are unchanged, so the same rule instance is again valid and yields a derivation d' in $\mathit{NMMS}(\mid\!\sim_0)$. By construction, $\mathit{Tree}(d')$ is precisely the tree obtained from $\mathit{Tree}(d_{\text{out}})$ by grafting u at the leaf a. Hence the grafted tree is again the proof tree of a derivation. □

Corollary 39. *If u and t are proof trees arising from derivations, then every element of $\mathit{Graft}(u, t)$ also arises from a derivation.*

Proof. Apply the previous proposition to each graftable leaf. □

5 The raw grafting product and graft-pair decomposition

Let $\mathbb{Z}\langle \mathit{PTree} \rangle$ be the free abelian group on proof trees.

Definition 40 (Linear proof-tree carrier)**.** *Equivalently,*

$$\mathbb{Z}\langle \mathit{PTree} \rangle = \{\, f : \mathit{PTree} \to \mathbb{Z} \mid \operatorname{supp}(f) \text{ is finite} \,\},$$

with pointwise addition and scalar multiplication. For each tree $t \in \mathit{PTree}$ we write e_t for the characteristic function of $\{t\}$, so that every element of $\mathbb{Z}\langle \mathit{PTree} \rangle$ has a unique expression

$$\sum_{t \in \mathit{PTree}} n_t e_t$$

with only finitely many nonzero coefficients $n_t \in \mathbb{Z}$.

Definition 41. *For proof trees u, t, define*

$$u \triangleright t := \sum_{a \in \mathrm{Leaf}_u(t)} e_{\gamma_{u,t}(a)}$$

and extend this bilinearly to $\mathbb{Z}\langle PTree \rangle$.

Definition 42 (Raw grafting operator)**.** *Equivalently, the raw product is the bilinear operator*

$$m_{\mathrm{raw}} : \mathbb{Z}\langle PTree \rangle \otimes_{\mathbb{Z}} \mathbb{Z}\langle PTree \rangle \longrightarrow \mathbb{Z}\langle PTree \rangle$$

determined on basis elements by

$$m_{\mathrm{raw}}(e_u \otimes e_t) = \sum_{a \in \mathrm{Leaf}_u(t)} e_{\gamma_{u,t}(a)}.$$

Thus $u \triangleright t$ is simply $m_{\mathrm{raw}}(e_u \otimes e_t)$, written in the proof-tree basis.

Proposition 43 (Multiplicity formula)**.** *For every $u, t, w \in PTree$, the coefficient of e_w in $u \triangleright t$ is exactly $m_{u,t}(w)$. Equivalently,*

$$u \triangleright t = \sum_{w \in \mathrm{SuppGraft}(u,t)} m_{u,t}(w)\, e_w.$$

Proof. By definition,

$$u \triangleright t = \sum_{a \in \mathrm{Leaf}_u(t)} e_{\gamma_{u,t}(a)}.$$

For fixed $w \in PTree$, the basis vector e_w occurs once for each address $a \in \mathrm{Leaf}_u(t)$ such that $\gamma_{u,t}(a) = w$. Hence its coefficient is exactly

$$\#\{\, a \in \mathrm{Leaf}_u(t) \mid \gamma_{u,t}(a) = w \,\} = m_{u,t}(w).$$

Collecting equal basis terms yields the second displayed formula. □

Remark 44. *This is the proof-tree analogue of Grossman–Larson grafting: one inserts the rooted proof tree u at every matching leaf of t, but only when the sequent interface matches exactly. The operation therefore depends on proof structure, not merely on the underlying combinatorics of rooted trees.*

Remark 45. *The operation $\triangleright$ just defined is the* raw pre-quotient grafting product *on the free abelian group $\mathbb{Z}\langle PTree \rangle$. In the formal development one proves that this bilinear operation satisfies the right pre-Lie identity. What is mathematically important for the present paper is that the proof strategy analysed below does not proceed by a direct cardinality comparison of raw address witnesses. Instead, one studies the set of* two-step witnesses *for the associator, partitions that set by a proof-theoretically meaningful equivalence relation, and compares the resulting quotient classes. The quotient therefore does not define the raw product; it organizes the proof of the pre-Lie identity for that product.*

Theorem 46 (Raw right pre-Lie identity)**.** *The bilinear product $\triangleright$ on $\mathbb{Z}\langle PTree\rangle$ satisfies the right pre-Lie identity*

$$(a \triangleright b) \triangleright c - a \triangleright (b \triangleright c) = (a \triangleright c) \triangleright b - a \triangleright (c \triangleright b)$$

for all $a, b, c \in \mathbb{Z}\langle PTree\rangle$.

Proof. By bilinearity, it suffices to prove the identity on basis elements e_x, e_y, e_z, i.e. on proof trees $x, y, z \in PTree$. For fixed output tree w, the coefficient of e_w in each side is computed by counting the appropriate two-step witness classes contributing to the corresponding associator term. The quotient analysis developed below proves that these class contributions match output-by-output: the noise sector is quotient-trivial, the mixed residual sector is empty, and the pure residual sectors transport uniquely across the swapped bracketing. This is exactly the class-level pre-Lie comparison theorem formalized in the Lean development. Hence the coefficients of e_w agree for every w, so the two sides are equal in $\mathbb{Z}\langle PTree\rangle$. Bilinearity then yields the stated identity for arbitrary a, b, c. □

Remark 47. *The quotient fibres $\mathcal{Q}(x, y, z; w)$ introduced below are not the carrier of this algebra. They are finite output-indexed witness quotients used to prove the preceding theorem by organizing the two-step associator comparison class-by-class.*

5.1 Ordered pairs of grafts

Fix proof trees x, y, z, w. An *ordered pair of grafts from (x, y, z) to w* consists of a first graft of y into z at a matching leaf yielding an intermediate tree z', followed by a second graft of x into z' yielding w.

Definition 48 (Outer and inner two-step witnesses)**.** *Let $a \in Addr(z)$ be the address of the first graft and let $z' = \mathrm{gr}_a(y, z)$. Since the same address a names the root of the inserted copy of y inside z', a second graft at address $b \in Addr(z')$ is called:*

- inner *if $a \preceq b$, equivalently if there exists $c \in Addr(y)$ with $b = a \cdot c$;*
- outer *if a and b are incomparable in the prefix order on $Addr(z')$.*

Thus outer and inner are defined entirely by the partial order $(Addr(z'), \preceq)$ on addresses.

Lemma 49 (Address classification after grafting)**.** *Let y be grafted into z at address a, yielding z'. Let x be successfully grafted into z' at address b. Then either*

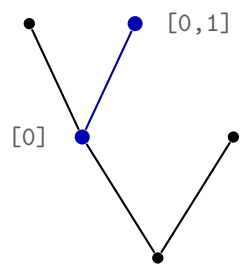
(i) *$b = a \cdot c$ for some non-empty extension c, so the second graft site lies inside the subtree introduced by the first; or*

(ii) *a and b are incomparable, so the two graft sites refer to disjoint parts of the tree.*

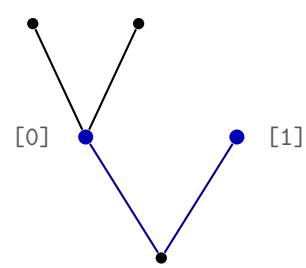
In particular, the case where b is a strict prefix of a — i.e. the second graft would occur at an ancestor of the first graft site — is impossible.

Proof sketch. The impossible case is eliminated by the sequent-interface constraint. If b were a strict prefix of a, then a would extend b, meaning a points into the subtree rooted at address b in z'. But address b in z' is a graftable leaf — a leaf carrying the conclusion of x. No address strictly extending a leaf address can point to a valid subtree, since leaves have no children. This contradiction is the proof-theoretically essential step: it is precisely the rigidity of the leaf-matching constraint, which forbids grafting at a non-leaf position, that eliminates the prefix case. The argument is formalized as `graftMatchingLeafAt_address_classification` in the Lean development. □

Remark 50. *This lemma is the geometric core on which the graft-pair decomposition rests. Its proof depends essentially on the fact that grafting operates on leaves with exact sequent labels, not on arbitrary internal vertices. In a setting with weakening, the leaf-interface constraint would be relaxed and this argument would not go through in the same form. The lemma is therefore sensitive to the structural rule choices made in Section 2.*



comparable
`[0]` is ancestor of `[0,1]`
(inner case)



incomparable
neither is ancestor of the other
(outer case)

Blue marks the pair of graft addresses under comparison. Left: `[0,1]` extends `[0]`, so the second graft site lies *inside* the subtree rooted at the first site — the inner (nested) case. Right: `[0]` and `[1]` are on disjoint branches — the outer (incomparable-address) case, where the order of insertion can be reversed.

Theorem 51 (Graft-pair decomposition)**.** *Let $a \in \mathrm{Leaf}_y(z)$ and $b \in \mathrm{Leaf}_x(z')$ and suppose*

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_b(x, z') = w.$$

Then exactly one of the following holds:

(i) Inner case: *there exists a nonempty address $c \in Addr(y) \setminus \{\langle\rangle\}$ such that*

$$b = a \cdot c.$$

Equivalently,

$$b \in Addr_{z'}(a) \setminus \{a\}.$$

If $y' = \mathrm{gr}_c(x, y)$*, then*

$$\mathrm{gr}_{a \cdot c}(x, z') = \mathrm{gr}_a(y', z).$$

(ii) Outer case: *a and b are incomparable in the partially ordered set* $(Addr(z'), \preceq)$*, and there exists* $z_3 \in PTree$ *such that*

$$\mathrm{gr}_b(x, z) = z_3 \qquad \textit{and} \qquad \mathrm{gr}_a(y, z_3) = w.$$

Proof. Let a be the address at which y is grafted into z, and let b be the address at which x is grafted into the resulting tree z'. By the address-classification theorem formalized in `two_step_graft_decomposition_full`, exactly one of two things can happen.

Either $a \prec b$. By definition of the prefix order there is a nonempty sequence c such that $b = a \cdot c$. Equivalently,

$$b \in Addr_{z'}(a) \setminus \{a\}.$$

Thus the second graft address lies in the address cone below the root of the inserted copy of y, so the second graft is inner. If y' is obtained from y by grafting x at c, then locality of replacement at a subtree gives

$$\mathrm{gr}_{a \cdot c}(x, (\mathrm{gr}_a(y, z))) = \mathrm{gr}_a((\mathrm{gr}_c(x, y)), z).$$

So the same output w may be read as arising from an inner graft of x into y, followed by insertion of the result into z. This is the inner case.

Or else $a \not\preceq b$. Since the second graft is successful, the address classification lemma excludes the possibility that $b \prec a$. Therefore a and b are incomparable in the partial order $(Addr(z'), \preceq)$. By the preceding proposition,

$$Addr_{z'}(a) \cap Addr_{z'}(b) = \varnothing.$$

Hence the two local replacements are supported on disjoint rooted subtrees of z', and so the order of the two graft operations may be reversed without changing the resulting proof tree. This is the outer case.

The two cases are mutually exclusive because a pair of addresses in a partial order cannot be simultaneously comparable and incomparable. □

Remark 52. *This theorem is the geometric heart of the associator analysis. It shows that the only essential distinction among graft pairs is between the prefix-comparable and prefix-incomparable regimes for the two graft addresses.*

Theorem 53 (Commutation at incomparable addresses)**.** *If two successful graftings occur at incomparable addresses, then performing them in either order yields the same final proof tree.*

Proof. If addresses a and b are incomparable, then neither extends the other. Hence the subtree rooted at a and the subtree rooted at b are disjoint. Grafting at a replaces only the subtree at a, so it leaves the subtree at b unchanged; similarly, grafting at b leaves the subtree at a unchanged.

Therefore the two local replacement operations commute:

$$\mathrm{gr}_b(x, (\mathrm{gr}_a(y, z))) = \mathrm{gr}_a(y, (\mathrm{gr}_b(x, z))).$$

Whenever both successive graftings are defined, they yield the same final proof tree. □

6 Algebraic preliminaries

We now recall the standard algebraic notion to which the proof-theoretic analysis will later lead.

Definition 54. *A* right pre-Lie algebra *over $\mathbb{Z}$ is an abelian group A equipped with a bilinear product $\triangleright$ satisfying*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z) = (x \triangleright z) \triangleright y - x \triangleright (z \triangleright y)$$

for all $x, y, z \in A$.

Remark 55. *Every right pre-Lie algebra carries an induced Lie bracket*

$$[x, y] := x \triangleright y - y \triangleright x.$$

The present paper does not yet construct the full enveloping Hopf algebra from this bracket. Instead, it establishes the quotient-controlled proof-theoretic pre-Lie identity that would supply the input for such a construction.

Why quotienting is necessary. One might hope to verify the pre-Lie identity directly at the level of raw proof trees, by tracking explicit address data. This fails for a structural reason: a single term $x \triangleright (y \triangleright z)$ can be witnessed *by many distinct ordered pairs of graft addresses, depending on which matching leaf of $y \triangleright z$ is chosen for the outer step. These witnesses encode the same geometric configuration — the same insertion event at the same relative position in the tree — but differ in the raw list entry that records it. Any equation between graft pairs is therefore really an equation between* equivalence classes *of address records, not between the records themselves.*

More precisely, two witnesses are bureaucratically equivalent *if they agree on which insertion is nested inside which, and whether the outer insertions act on disjoint branches or on ancestor-descendant pairs. The quotient by this equivalence retains exactly the dependency geometry and discards all bookkeeping accidents. The pre-Lie identity*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z) = (x \triangleright z) \triangleright y - x \triangleright (z \triangleright y)$$

holds in this quotient: the left-hand side and the right-hand side collect the same *noise classes with opposite signs (those cancel) and the same residual classes (those match term for term). Without the quotient, the witness sets on the two sides of the equation are not equal as sets of address records, only equal in cardinality after noise removal; the quotient is what supplies the required bijection.*

For fixed x, y, z, w, let $\mathcal{W}_L(x, y, z; w)$ be the set of successful ordered pairs of graftings contributing to the coefficient of w in $(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z)$, and let $\mathcal{W}_R(x, y, z; w)$ be the analogous set for the swapped expression $(y \triangleright x) \triangleright z - y \triangleright (x \triangleright z)$. Then the raw pre-Lie identity would amount to a sign-preserving bijection between $\mathcal{W}_L(x, y, z; w)$ and $\mathcal{W}_R(x, y, z; w)$ for every output tree w. What the formal development shows is subtler: there is in general no canonical bijection at raw witness level, but there is a canonical comparison after passing to quotient classes of witnesses. On those quotient classes, the noise sector cancels and the residual sector is matched bijectionally.

7 Quotienting bureaucratic graft-pair data

At raw address level, graft-pair configurations still contain too much presentation data. The next step is to isolate their genuine dependency geometry.

7.1 Associator witnesses as ordered graft pairs

Definition 56. *Fix proof trees x, y, z, w. A* left witness for *w is a successful ordered pair of grafting operations contributing to the left-bracketed associator. Concretely, it is one of the following two kinds of data.*

(i) Outer witness: *addresses a, b and an intermediate tree z' such that grafting y into z at address a yields z', and then grafting x into z' at address b yields w:*

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_b(x, z') = w.$$

(ii) Inner witness: *an internal address c in x, an address a in z, and an intermediate tree u such that grafting y into x at address c yields u, and then grafting u into z at address a yields w:*

$$\mathrm{gr}_c(y, x) = u, \qquad \mathrm{gr}_a(u, z) = w.$$

A swapped-right witness for *w is defined analogously:*

(i) Outer witness: *addresses a, b and an intermediate tree z' such that*

$$\mathrm{gr}_a(x, z) = z', \qquad \mathrm{gr}_b(y, z') = w.$$

(ii) Inner witness: *an internal address c in y, an address a in z, and an intermediate tree u such that*

$$\mathrm{gr}_c(x, y) = u, \qquad \mathrm{gr}_a(u, z) = w.$$

Definition 57 (Witness sets)**.** *For fixed x, y, z, w, let $W_L^{\mathrm{out}}(x, y, z; w)$ be the set of all triples $(a, b, z') \in \mathbb{N}^{<\omega} \times \mathbb{N}^{<\omega} \times PTree$ such that $a \in Addr(z)$, $b \in Addr(z')$, and*

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_b(x, z') = w.$$

Let $W_L^{\mathrm{in}}(x, y, z; w)$ be the set of all tuples $(c, a, u) \in \mathbb{N}^{<\omega} \times \mathbb{N}^{<\omega} \times PTree$ such that $c \in Addr(x)$, $a \in Addr(z)$, and

$$\mathrm{gr}_c(y, x) = u, \qquad \mathrm{gr}_a(u, z) = w.$$

$W_R^{\mathrm{out}}(x, y, z; w) := \{(a, b, z') \in \mathbb{N}^{<\omega} \times \mathbb{N}^{<\omega} \times PTree \mid a \in Addr(z),\ b \in Addr(z'),\ \mathrm{gr}_a(x, z) = z',\ \mathrm{gr}_b(y, z') = w\}$,

$W_R^{\mathrm{in}}(x, y, z; w) := \{(c, a, u) \in \mathbb{N}^{<\omega} \times \mathbb{N}^{<\omega} \times PTree \mid c \in Addr(y),\ a \in Addr(z),\ \mathrm{gr}_c(x, y) = u,\ \mathrm{gr}_a(u, z) = w\}$.

The full left and swapped-right witness fibres are

$$W_L(x, y, z; w) := W_L^{\mathrm{out}}(x, y, z; w) \cup W_L^{\mathrm{in}}(x, y, z; w).$$

$$W_R(x, y, z; w) := W_R^{\mathrm{out}}(x, y, z; w) \cup W_R^{\mathrm{in}}(x, y, z; w).$$

Remark 58 (Set-theoretic form of the two-step dichotomy)**.** *The graft-pair decomposition theorem proved earlier implies that every successful ordered pair of grafts contributing to the left bracketing belongs to exactly one of the two fibres $W_L^{\mathrm{out}}(x, y, z; w)$ or $W_L^{\mathrm{in}}(x, y, z; w)$; likewise, every successful ordered pair contributing to the swapped-right bracketing belongs to exactly one of $W_R^{\mathrm{out}}(x, y, z; w)$ or $W_R^{\mathrm{in}}(x, y, z; w)$. Thus the outer/inner distinction is defined before quotienting; the quotient is used only to identify different presentations of the same two-step geometry.*

7.2 Graft-pair classes

Definition 59 (Tagged raw code fibres)**.** *For fixed x, y, z, w, define*

$$\mathrm{Code}(x, y, z; w) := \big(\{\mathsf{Lout}\} \times W_L^{\mathrm{out}}(x, y, z; w)\big) \sqcup \big(\{\mathsf{Lin}\} \times W_L^{\mathrm{in}}(x, y, z; w)\big)$$
$$\sqcup \big(\{\mathsf{Rout}\} \times W_R^{\mathrm{out}}(x, y, z; w)\big) \sqcup \big(\{\mathsf{Rin}\} \times W_R^{\mathrm{in}}(x, y, z; w)\big).$$

We write the corresponding tagged elements as

$$\mathsf{Lout}(a, b, z'), \qquad \mathsf{Lin}(c, a, u), \qquad \mathsf{Rout}(a, b, z'), \qquad \mathsf{Rin}(c, a, u).$$

Definition 60 (Generator relations)**.** *Fix x, y, z, w. The four basic comparison relations on $\mathrm{Code}(x, y, z; w)$ are the following subsets of $\mathrm{Code}(x, y, z; w) \times \mathrm{Code}(x, y, z; w)$.*

(i) *R_{oo} consists of all pairs*

$$\big(\mathsf{Lout}(a, b, z'), \mathsf{Rout}(b, a, z_3)\big)$$

such that $(a, b, z') \in W_L^{\mathrm{out}}(x, y, z; w)$, both a and b belong to $Addr(z)$, a and b are incomparable in $(Addr(z), \preceq)$, $z_3 = \mathrm{gr}_b(x, z)$, and

$$\mathrm{gr}_a(y, z_3) = w.$$

(ii) R_{oi} *consists of all pairs*

$$\big(\mathsf{Lout}(a, a \cdot c, z'), \mathsf{Rin}(c, a, u)\big)$$

such that $(a, a \cdot c, z') \in W_L^{\mathrm{out}}(x, y, z; w)$, $c \neq \langle\rangle$,

$$u = \mathrm{gr}_c(x, y), \qquad \mathrm{gr}_a(u, z) = w.$$

(iii) R_{bo} *consists of all pairs*

$$\big(\mathsf{Rout}(a, b, z'), \mathsf{Lout}(b, a, z_3)\big)$$

such that $(a, b, z') \in W_R^{\mathrm{out}}(x, y, z; w)$, *both* a *and* b *belong to* $Addr(z)$, a *and* b *are incomparable in* $(Addr(z), \preceq)$, $z_3 = \mathrm{gr}_b(y, z)$, *and*

$$\mathrm{gr}_a(x, z_3) = w.$$

(iv) R_{bi} *consists of all pairs*

$$\big(\mathsf{Rout}(a, a \cdot c, z'), \mathsf{Lin}(c, a, u)\big)$$

such that $(a, a \cdot c, z') \in W_R^{\mathrm{out}}(x, y, z; w)$, $c \neq \langle\rangle$,

$$u = \mathrm{gr}_c(y, x), \qquad \mathrm{gr}_a(u, z) = w.$$

Definition 61 (Bureaucratic equivalence)**.** *The* bureaucratic equivalence relation

$$\sim_{\mathrm{b}} \subseteq \mathrm{Code}(x, y, z; w) \times \mathrm{Code}(x, y, z; w)$$

is the least equivalence relation containing the four generator relations

$$R_{\mathrm{oo}}, \qquad R_{\mathrm{oi}}, \qquad R_{\mathrm{bo}}, \qquad R_{\mathrm{bi}}.$$

Proposition 62 (Bureaucratic equivalence preserves the composite tree)**.** *Let* $\xi, \eta \in \mathrm{Code}(x, y, z; w)$.

(i) *If* $(\xi, \eta) \in R_{\mathrm{oo}} \cup R_{\mathrm{oi}} \cup R_{\mathrm{bo}} \cup R_{\mathrm{bi}}$, *then* ξ *and* η *determine the same terminal proof tree* w.

(ii) *If* $\xi \sim_{\mathrm{b}} \eta$, *then* ξ *and* η *represent the same class in the quotient* $\mathrm{Code}(x, y, z; w)/{\sim_{\mathrm{b}}}$.

Proof. For (i), suppose first that

$$\big(\mathsf{Lout}(a, b, z'), \mathsf{Rout}(b, a, z_3)\big) \in R_{\mathrm{oo}}.$$

Then

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_b(x, z') = w, \qquad \mathrm{gr}_b(x, z) = z_3,$$

and $a, b \in Addr(z)$ are incomparable. By the commutation theorem for graftings at incomparable addresses,

$$\mathrm{gr}_a(y, z_3) = w.$$

Hence the two tagged codes lie in the same fibre and have the same output.

Suppose next that

$$\big(\mathsf{Lout}(a, a\cdot c, z'), \mathsf{Rin}(c, a, u)\big) \in R_{\mathrm{oi}}.$$

Then

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_{a\cdot c}(x, z') = w, \qquad u = \mathrm{gr}_c(x, y), \qquad \mathrm{gr}_a(u, z) = w.$$

Moreover,

$$\mathrm{gr}_{a\cdot c}(x, z') \;=\; \mathrm{gr}_{a\cdot c}(x, \mathrm{gr}_a(y, z)) \;=\; \mathrm{gr}_a(\mathrm{gr}_c(x, y), z) \;=\; \mathrm{gr}_a(u, z),$$

which is exactly the locality-of-replacement identity on the rooted subtree inserted at address a. Thus the two tagged codes again have the same terminal tree w.

The cases of R_{bo} and R_{bi} are the same two arguments with x and y interchanged.

For (ii), equality of quotient classes is by definition the equivalence relation $\sim_{\mathrm{b}}$. Thus $\xi \sim_{\mathrm{b}} \eta$ implies

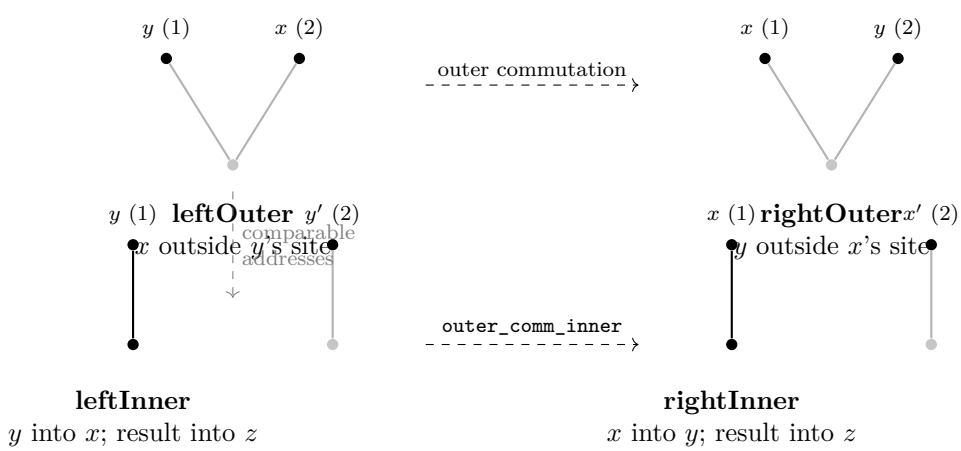
$$[\,\xi\,] = [\,\eta\,] \qquad \text{in} \qquad \mathrm{Code}(x, y, z; w)/{\sim_{\mathrm{b}}}.$$

□

Remark 63. *The quotient by $\sim_{\mathrm{b}}$ is therefore a quotient of the tagged raw code fibre* $\mathrm{Code}(x, y, z; w)$*, not of the set of trees with terminal label w. Equality in the quotient is equality of equivalence classes of two-step presentations:*

$$[\,\xi\,] = [\,\eta\,] \quad \Longleftrightarrow \quad \xi \sim_{\mathrm{b}} \eta.$$

What is forgotten is the particular address presentation of the two-step composite; what is retained is the quotient class of that presentation.



This figure is schematic. Its four labelled positions represent the four tagged components Lout, Lin, Rout, Rin of the fibre $\mathrm{Code}(x, y, z; w)$. The upper dashed arrow represents the relation $R_{\mathrm{oo}} \subseteq \mathsf{Lout} \times \mathsf{Rout}$, defined on incomparable address pairs in $Addr(z)$. The comparable-address transport is the relation $R_{\mathrm{oi}} \subseteq \mathsf{Lout} \times \mathsf{Rin}$, defined on tuples of the form $(a, a \cdot c, z')$. The converse relations are $R_{\mathrm{bo}} \subseteq \mathsf{Rout} \times \mathsf{Lout}$ and $R_{\mathrm{bi}} \subseteq \mathsf{Rout} \times \mathsf{Lin}$. Grey vertices belong to the receiver tree z; black vertices indicate the address components occurring in the corresponding raw code tuples.

Proposition 64 (Comparable-address criterion for the mixed generator)**.** *Let*

$$\xi = \mathsf{Lout}(a, b, z') \in \mathrm{Code}(x, y, z; w).$$

Then there exists $\eta \in \mathrm{Code}(x, y, z; w)$ *such that*

$$(\xi, \eta) \in R_{\mathrm{oi}}$$

if and only if there exists an address $c \in \mathbb{N}^{<\omega}$ *with*

$$b = a \cdot c.$$

Whenever this holds, every such η *has the form*

$$\eta = \mathsf{Rin}(c, a, u)$$

for a tree u *satisfying*

$$u = \mathrm{gr}_c(x, y), \qquad \mathrm{gr}_a(u, z) = w.$$

Proof. If $(\xi, \eta) \in R_{\mathrm{oi}}$, then by definition of R_{oi} ξ must be of the form $\mathsf{Lout}(a, a \cdot c, z')$ for some $c \in \mathbb{N}^{<\omega}$. Thus $b = a \cdot c$.

Conversely, suppose $b = a \cdot c$. If

$$\xi = \mathsf{Lout}(a, a \cdot c, z')$$

lies in $\mathrm{Code}(x, y, z; w)$, then

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_{a \cdot c}(x, z') = w.$$

Set $u := \mathrm{gr}_c(x, y)$. By locality of replacement on the rooted subtree inserted at a,

$$\mathrm{gr}_a(u, z) \;=\; \mathrm{gr}_a(\mathrm{gr}_c(x, y), z) \;=\; \mathrm{gr}_{a \cdot c}(x, \mathrm{gr}_a(y, z)) \;=\; \mathrm{gr}_{a \cdot c}(x, z') \;=\; w.$$

Therefore

$$\eta := \mathsf{Rin}(c, a, u)$$

satisfies $(\xi, \eta) \in R_{\mathrm{oi}}$. The final statement is again just the defining form of R_{oi}. $\square$

Proposition 65 (Identity of two-step proof compositions)**.** *For $c, d \in \mathrm{Code}(x, y, z; w)$,*

$$[c] = [d] \text{ in } \mathrm{Code}(x, y, z; w)/\sim_{\mathrm{b}}$$

if and only if there exists a finite chain

$$c = c_0,\ c_1,\ \ldots,\ c_n = d$$

of raw graft-pair codes in $\mathrm{Code}(x, y, z; w)$ such that, for each $i < n$, the pair (c_i, c_{i+1}) is related by one generating move of the bureaucratic equivalence relation, that is, belongs to

$$R_{\mathrm{oo}} \cup R_{\mathrm{oi}} \cup R_{\mathrm{bo}} \cup R_{\mathrm{bi}}$$

or to the converse of one of these four relations.

Proof. By definition, $\mathcal{Q}(x, y, z; w)$ is the quotient of $\mathrm{Code}(x, y, z; w)$ by the equivalence relation $\sim_{\mathrm{b}}$ generated by the four relations

$$R_{\mathrm{oo}}, \qquad R_{\mathrm{oi}}, \qquad R_{\mathrm{bo}}, \qquad R_{\mathrm{bi}}.$$

Hence $[c] = [d]$ is equivalent to $c \sim_{\mathrm{b}} d$. Since $\sim_{\mathrm{b}}$ is the least equivalence relation containing those four relations, the statement $c \sim_{\mathrm{b}} d$ is in turn equivalent to the existence of a finite zig-zag from c to d built from generators and their converses. This is exactly the required chain description. □

Definition 66. *The quotient set of graft-pair codes is denoted*

$$\mathcal{Q}(x, y, z; w).$$

More formally, if $\mathrm{Code}(x, y, z; w)$ denotes the set of all raw graft-pair codes with output w, and if $\sim_{\mathrm{b}}$ is the bureaucratic equivalence relation, then

$$\mathcal{Q}(x, y, z; w) := \mathrm{Code}(x, y, z; w)/\sim_{\mathrm{b}}.$$

Inside it we distinguish the left contribution classes

$$\mathcal{C}_{\mathrm{L}}(x, y, z; w)$$

and the swapped-right contribution classes

$$\mathcal{C}_{\mathrm{R}}(x, y, z; w),$$

the latter being associated with the swapped bracketing (y, x, z).

Definition 67 (Quotient projection)**.** *The quotient map*

$$\pi_{x,y,z;w} : \mathrm{Code}(x, y, z; w) \twoheadrightarrow \mathcal{Q}(x, y, z; w)$$

sends a raw graft-pair code c to its bureaucratic equivalence class $[c]$. In particular, the left and swapped-right contribution classes are images under $\pi_{x,y,z;w}$ of the corresponding subsets of raw left and raw swapped-right witnesses.

Definition 68 (Classwise comparison relation)**.** *The relation*

$$\mathsf{S}_{x,y,z;w} \subseteq \mathcal{Q}(x,y,z;w) \times \mathcal{Q}(y,x,z;w)$$

is defined by declaring $([c],[d]) \in \mathsf{S}_{x,y,z;w}$ *when there exist raw representatives*

$$\xi \in \mathrm{Code}(x,y,z;w), \qquad \eta \in \mathrm{Code}(y,x,z;w)$$

with

$$\pi_{x,y,z;w}(\xi) = [c], \qquad \pi_{y,x,z;w}(\eta) = [d],$$

such that the ordered pair (ξ,η) *lies in the graph of one of the four generator relations after swapping the parameters* $(x,y) \leftrightarrow (y,x)$*. Equivalently,* $\mathsf{S}_{x,y,z;w}$ *is the quotient image of the union of the four generator relations under the two quotient maps. Because these relations are compatible with* $\sim_{\mathrm{b}}$*,* $\mathsf{S}_{x,y,z;w}$ *is well-defined.*

Remark 69. *An element of* $\mathcal{C}_{\mathrm{L}}(x,y,z;w)$ *or* $\mathcal{C}_{\mathrm{R}}(x,y,z;w)$ *should therefore be read as an equivalence class of* pairs of grafting operations *with output* w*, not as a single derivation history. The quotient identifies histories that differ only by bureaucratic scheduling, or by two equivalent descriptions of the same nested dependency. This is the formal counterpart of the proof-theoretic principle that identity of proof should ignore inessential orderings of operations supported at incomparable addresses while retaining genuine dependency.*

Definition 70 (Support predicates and sector decomposition on the left carrier)**.** *Fix* x,y,z,w*, and let* $q \in \mathcal{C}_{\mathrm{L}}(x,y,z;w)$*.*

(i) q *has* left outer support *if it admits a representative in* $W_L^{\mathrm{out}}(x,y,z;w)$*.*

(ii) q *has* left inner support *if it admits a representative in* $W_L^{\mathrm{in}}(x,y,z;w)$*.*

(iii) q *has* right outer support *if there exists* $q' \in \mathcal{C}_{\mathrm{R}}(x,y,z;w)$ *such that* $(q,q') \in \mathsf{S}_{x,y,z;w}$ *and* q' *admits a representative in* $W_R^{\mathrm{out}}(x,y,z;w)$*.*

(iv) q *has* right inner support *if there exists* $q' \in \mathcal{C}_{\mathrm{R}}(x,y,z;w)$ *such that* $(q,q') \in \mathsf{S}_{x,y,z;w}$ *and* q' *admits a representative in* $W_R^{\mathrm{in}}(x,y,z;w)$*.*

The two quotient-trivial noise sectors on the left carrier are then

$$\mathsf{Noise}_{L\to R}(x,y,z;w) := \{\, q \in \mathcal{C}_{\mathrm{L}}(x,y,z;w) \mid q \text{ has left outer support and right inner support} \,\},$$

$$\mathsf{Noise}_{R\to L}(x,y,z;w) := \{\, q \in \mathcal{C}_{\mathrm{L}}(x,y,z;w) \mid q \text{ has right outer support and left inner support} \,\}.$$

The residual left sector *is*

$$\mathsf{Res}_L(x,y,z;w) := \mathcal{C}_{\mathrm{L}}(x,y,z;w) \setminus \big(\mathsf{Noise}_{L\to R}(x,y,z;w) \cup \mathsf{Noise}_{R\to L}(x,y,z;w)\big).$$

Finally, for $q \in \mathsf{Res}_L(x,y,z;w)$*, define*

$$q \in \mathsf{PureOut}_L(x,y,z;w) \iff q \text{ has left outer support and not left inner support},$$

$$q \in \mathsf{PureIn}_L(x,y,z;w) \iff q \text{ has left inner support and not left outer support},$$

$$q \in \mathsf{Mixed}_L(x,y,z;w) \iff q \text{ has both left outer and left inner support.}$$

Remark 71 (Mathematical form of the quotient-level comparison)**.** *For fixed x, y, z, w, the class-level pre-Lie comparison problem is the comparison of the two quotient carriers $\mathcal{C}_{\mathrm{L}}(x, y, z; w)$ and $\mathcal{C}_{\mathrm{R}}(x, y, z; w)$ through the transport relation $\mathsf{S}_{x,y,z;w}$. On the left carrier, the quotient-trivial part is exactly*

$$\mathsf{Noise}_{L\to R}(x, y, z; w) \cup \mathsf{Noise}_{R\to L}(x, y, z; w),$$

while the nontrivial part is the residual set $\mathsf{Res}_L(x, y, z; w)$. The later theorems prove:

(i) *every residual left class belongs to exactly one of the three sectors $\mathsf{PureOut}_L(x, y, z; w)$, $\mathsf{PureIn}_L(x, y, z; w)$, $\mathsf{Mixed}_L(x, y, z; w)$;*

(ii) *the mixed residual sector $\mathsf{Mixed}_L(x, y, z; w)$ is empty;*

(iii) *the outer and inner transport maps induce unique partners for classes in $\mathsf{PureOut}_L(x, y, z; w)$ and $\mathsf{PureIn}_L(x, y, z; w)$, respectively.*

Thus the class-level associator comparison reduces to equality on the quotient-trivial noise sector and to unique transport on the two pure residual sectors.

7.3 Canonical transport theorems

Theorem 72 (Outer transport)**.** *For every x, y, z, w, outer left contribution classes are canonically in bijection with outer contribution classes on the swapped right side.*

Proof. Take an outer left witness represented by a pair of successful grafts

$$\mathrm{gr}_a(y, z) = z', \qquad \mathrm{gr}_b(x, z') = w,$$

with a and b incomparable. By commutation at incomparable addresses, there is a unique tree z'' such that

$$\mathrm{gr}_b(x, z) = z'', \qquad \mathrm{gr}_a(y, z'') = w.$$

This defines a swapped right-outer witness with the same output w.

Because this construction is obtained by commuting the same two local operations, it depends only on the dependency geometry of the witness and not on the particular address record chosen inside its bureaucratic equivalence class. Hence it descends to quotient classes and gives a map from outer left classes to swapped right-outer classes.

Applying exactly the same commutation in the reverse direction recovers the original witness. The two maps are therefore inverse to one another, so we obtain a canonical bijection of outer contribution classes. □

Theorem 73 (Inner transport)**.** *For every x, y, z, w, inner left contribution classes are canonically in bijection with inner contribution classes on the swapped right side.*

Proof. Take an inner left witness. By definition, it is specified by an internal address c in x, an address a in z, and an intermediate tree $u = \mathrm{gr}_c(y, x)$ such that

$$\mathrm{gr}_a(u, z) = w.$$

Now consider the swapped parameter order $(y, x, z; w)$. Relative to that order, the same triple (c, a, u) is a right-inner witness, because the first grafting operation is precisely

$$\mathrm{gr}_c(y, x) = u, \qquad \mathrm{gr}_a(u, z) = w.$$

Thus the inner transport is not a change in the underlying composite tree, but only a change in which ordered pair (x, y) is regarded as the outer bracketing parameter. The construction preserves the terminal tree w, depends only on the raw witness data (c, a, u), and is inverted by the same reinterpretation with the parameters swapped back. Therefore it descends to a canonical bijection between left-inner classes for $(x, y, z; w)$ and right-inner classes for $(y, x, z; w)$. □

Remark 74. *The outer transport is a strict commutation theorem. The inner transport is a reassociation theorem. Their common role is to show that the quotient keeps the geometric content of the graft pair while discarding the accidental presentation details.*

8 Noise, residual classes, and purity

8.1 Noise and residual sectors

Definition 75. *A left contribution class is called* noise *if the apparent difference between its outer and inner presentations is already quotient-trivial under the comparison with the swapped side. A left contribution class is* residual *if it is not noise.*

Remark 76. *"Noise" does not mean mathematically uninteresting in itself. It means precisely that the class contributes no genuine asymmetry to the associator comparison, because the quotient already identifies the competing presentations.*

Remark 77. *The geometric mechanism producing noise is the following. A left-outer contribution class becomes noise when it simultaneously admits a right-inner presentation. By the* `outer_comm_inner` *identification in the bureaucratic equivalence, this happens exactly when the two graft addresses are comparable. In that case, what appears from the left bracketing as an outer insertion of x into z' at an address incomparable with the first graft site can equally be read, from the right bracketing, as x being composed into y first and the result placed into z. The class thus contributes equally to both sides of the pre-Lie associator and cancels. This mechanism — comparable addresses producing simultaneously-outer-and-inner classes — is what the formal development calls* left-to-right overlap noise*. The noise sector is therefore not an arbitrary algebraic device but a precise reflection of a geometric degeneracy in the graft addresses.*

Definition 78. *A residual left contribution class is called:*

- pure outer *if it has outer support and no inner support;*
- pure inner *if it has inner support and no outer support;*
- mixed *if it has both outer and inner support.*

Theorem 79 (Elimination of mixed residual classes)**.** *There are no mixed residual left contribution classes.*

Lemma 80 (Mixed support forces overlap noise)**.** *Let $q \in \mathcal{C}_{\mathrm{L}}(x, y, z; w)$. If q has both left outer support and left inner support, then $q \in \mathsf{Noise}_{L\to R}(x, y, z; w)$.*

Proof. Choose a left-outer representative ξ of q and a left-inner representative η of q. Since $[\xi] = [\eta]$ in $\mathcal{Q}(x, y, z; w)$, Proposition 79 gives a finite chain of generating moves from ξ to η. Along such a chain, the only generator relation that changes a left-outer code into an inner code on the swapped side is R_{oi}, whose defining criterion is the comparable-address condition $b = a \cdot c$ with $c \neq \langle\rangle$. Hence some outer representative of q lies in the comparable-address regime.

By Proposition 78, such a comparable left-outer representative determines a right-inner representative of the same quotient class. Therefore q has left outer support and right inner support, so

$$q \in \mathsf{Noise}_{L\to R}(x, y, z; w).$$

□

Proof. Assume toward a contradiction that q is a mixed residual left contribution class. By definition of mixedness, q has both left outer support and left inner support. By the preceding lemma,

$$q \in \mathsf{Noise}_{L\to R}(x, y, z; w).$$

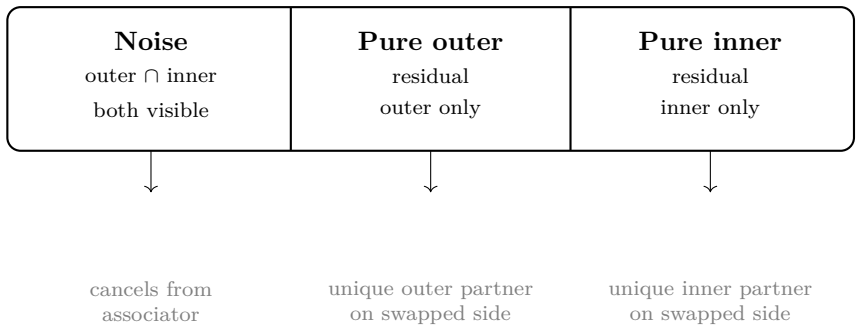
But q was assumed residual, so by definition

$$q \notin \mathsf{Noise}_{L\to R}(x, y, z; w) \cup \mathsf{Noise}_{R\to L}(x, y, z; w),$$

a contradiction. Therefore no mixed residual left contribution class exists. □

Corollary 81 (Residual purity)**.** *Every residual left contribution class is either pure outer or pure inner.*

Proof. By the theorem, the mixed case is impossible. □



The three sectors partition the left contribution classes at any fixed output w. Noise classes carry both outer and inner support and cancel from the associator by the `outer_comm_inner` identification. The elimination of mixed residual classes (proved above) ensures that the pure outer and pure inner sectors are exhaustive among residual classes. Each residual class then has a unique partner on the swapped right side, yielding the class-level pre-Lie identity.

Remark 82. *This is the decisive structural step in the whole proof. Once mixed residual classes disappear, the residual sector becomes typed and rigid, and the final pre-Lie comparison reduces to separate outer and inner transport theorems.*

8.2 Unique typed partners

Theorem 83 (Unique typed partner theorem)**.** *Let $q \in \mathcal{C}_{\mathrm{L}}(x, y, z; w)$ be residual.*

(i) *If q is pure outer, then there exists a unique pure outer class $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ corresponding to q.*

(ii) *If q is pure inner, then there exists a unique pure inner class $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ corresponding to q.*

Proof. Suppose first that q is pure outer. Then q has outer support, so the outer transport theorem produces a swapped right-outer class q' corresponding to q. Uniqueness among right-outer classes is built into the same transport theorem, since the transport maps are mutually inverse on the outer-supported sector.

It remains to check that the transported class is again pure outer on the swapped side. If it were noise, then q itself would already belong to the noise sector, because noise is exactly the quotient-trivial part of the comparison relation. That contradicts residuality. If instead q' carried an inner presentation as well, then transporting that inner information back to the left would furnish both outer and inner support for q, making q a mixed residual class. This is impossible by elimination of mixed residual classes. Hence q' is pure outer.

The proof for pure inner classes is the same, using the inner transport theorem in place of the outer one. Starting from a pure inner residual class q, the inner transport map produces a unique swapped right-inner class q'. Residual classes cannot

transport to noise, and if q' also carried outer support, transporting back would make q mixed residual, again impossible. Thus q' is pure inner, and uniqueness follows from the inverse inner transport. □

Theorem 84 (Residual classes seen from the swapped side)**.** *Every residual swapped-right contribution class arises from a pure residual left contribution class.*

Proof. Let q' be a residual swapped-right contribution class. The class-level decomposition on the swapped side says that every swapped-right contribution class is either noise or residual with a left partner. Since q' is assumed residual, the noise alternative is excluded. Therefore there exists a left contribution class q corresponding to q' under the swapped comparison relation.

Moreover, the decomposition theorem identifies such a left partner as residual. By residual purity, every residual left class is either pure outer or pure inner. Hence q is a pure residual left class, as required. □

9 The class-level pre-Lie identity

9.1 Associator contributions at a fixed output

Definition 85. *For proof trees x, y, define the* primitive class-level product support *by*

$$\mathsf{Prod}(x, y) := \{\, w \in \mathit{PTree} : w \in \mathit{Graft}(x, y) \,\}.$$

Remark 86. *This is not yet a quotient object. It is simply the set of outputs occurring in one-step grafting. The quotient enters only in the analysis of how these outputs receive graft-pair associator contributions.*

Definition 87. *Fix x, y, z, w. The* class-level associator at output w *is the assertion that left contribution classes to w decompose into noise, pure outer residual classes, and pure inner residual classes, and that these three sectors compare with the swapped side exactly as described by the noise, outer-transport, and inner-transport theorems.*

Theorem 88 (Class-level associator shape)**.** *For every x, y, z, w, the class-level associator at output w holds.*

Proof. Fix x, y, z, w. Let q be a left contribution class at output w. By the noise/residual decomposition, q is either noise or residual. If q is residual, then by residual purity it is either pure outer or pure inner. Thus every left contribution class belongs to one of the three sectors appearing in the definition of the class-level associator.

Next consider how these sectors compare with the swapped side. Noise classes are harmless by definition: they are exactly those classes whose left/right discrepancy is already quotient-trivial, so they contribute no genuine asymmetry to the associator. If q is pure outer residual, then by the unique typed partner theorem

it has a unique pure outer partner on the swapped side. If q is pure inner residual, the same theorem gives a unique pure inner partner on the swapped side.

Finally, let q' be a swapped-right contribution class. If q' is residual, then by the preceding theorem it arises from a pure residual left class. Thus no non-noise swapped-right class lies outside the pure residual image of the left side. This is exactly the structural assertion encoded in the definition of the class-level associator at w. □

Theorem 89 (Class-level pre-Lie comparison)**.** *Fix x, y, z, w. Every left contribution class to the coefficient of w in the associator belongs to exactly one of the following three sectors:*

(i) *quotient-trivial noise;*

(ii) *pure outer residual, with a unique swapped pure outer partner;*

(iii) *pure inner residual, with a unique swapped pure inner partner.*

Proof. Let q be a left contribution class contributing to the coefficient of w. By the class-level associator shape just proved, q lies in one of three mutually exclusive situations.

If q is noise, then it belongs to sector (i), and its contribution is already quotient-trivial. If q is pure outer residual, then the unique typed partner theorem gives a unique swapped pure outer partner, so q belongs to sector (ii). If q is pure inner residual, the same theorem gives a unique swapped pure inner partner, so q belongs to sector (iii).

These three cases exhaust all left contribution classes, and residual purity ensures that sectors (ii) and (iii) do not overlap. Thus the comparison theorem is exactly the operational form of the associator-shape theorem. □

9.2 The main theorem

Theorem 90 (Class-level pre-Lie identity)**.** *For all proof trees x, y, z, the class-level associator is symmetric under interchange of x and y. Equivalently, for every output tree w, the contribution classes to*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z)$$

are matched, after quotienting by bureaucratic equivalence, with the contribution classes to

$$(y \triangleright x) \triangleright z - y \triangleright (x \triangleright z).$$

Proof. Fix w. By the class-level pre-Lie comparison theorem, every left contribution class lies in exactly one of three sectors.

Noise. Noise classes are quotient-trivial by construction and contribute no genuine asymmetry.

Pure outer residual classes. These are matched uniquely across the two bracketings by outer transport.

Pure inner residual classes. These are matched uniquely across the two bracketings by inner transport.

Since these three sectors exhaust the left contribution classes at output w, the class-level associator at w is symmetric under swapping x and y. As w was arbitrary, the class-level pre-Lie identity follows. □

Remark 91. *This theorem is the precise endpoint currently established by the formal development. It is a statement about quotient-invariant associator comparison at the level of contribution classes. It does not yet claim a fully elaborated Hopf algebra in the main body of the paper.*

9.3 Further formally established consequences

The Lean development proves several sharper statements than are strictly needed for the narrative route to the main class-level pre-Lie theorem above. We record the most mathematically relevant ones here, because they clarify *where* the quotient enters and *what* has already been proved independently of the unfinished Hopf-algebra layer.

Definition 92. *For proof trees x and y, let*

$$\mathrm{Supp}(x, y) := \{\, w \in PTree \mid w \text{ occurs as a one-step graft output of } x \text{ into } y \,\}.$$

Equivalently, $\mathrm{Supp}(x, y)$ *is the support set of the primitive class-level product.*

Proposition 93 (Finite one-step support)**.** *For every pair of proof trees x, y, the set* $\mathrm{Supp}(x, y)$ *is finite.*

Proof. Every one-step graft output is obtained by choosing a leaf of y whose interface matches the conclusion of x, and then replacing that leaf by the tree x. Since y has only finitely many leaves, there are only finitely many admissible one-step grafts of x into y. Hence the set of outputs $\mathrm{Supp}(x, y)$ is finite. □

Proposition 94 (Support bridge between raw linearization and class-level product)**.** *Let $\mathbb{Z}[PTree]$ be the free abelian group on proof trees, and let e_x, e_y denote the basis vectors corresponding to x and y. Then for every proof tree w,*

$$w \in \mathrm{supp}(e_x \triangleright e_y) \quad \Longleftrightarrow \quad w \in \mathrm{Supp}(x, y).$$

In particular, the quotient does not alter the one-step *output set of proof composition. Any obstruction to a raw pre-Lie law must therefore occur at the level of* two-step witness comparison*, i.e. in the identity criteria for derivation histories rather than in the existence of one-step composites.*

Proof. By construction of the raw bilinear grafting product on $\mathbb{Z}[PTree]$, the product $e_x \triangleright e_y$ is the formal sum of all one-step matching-leaf graft outputs of x into y, counted with multiplicity. Equivalently,

$$e_x \triangleright e_y = \sum_{u \in PTree} m_{x,y}(u)\, e_u,$$

where $m_{x,y}(u)$ is the number of admissible graftings producing the tree u. In particular, the coefficient of e_w is exactly the number of occurrences of w among the one-step outputs. Therefore

$$w \in \operatorname{supp}(e_x \triangleright e_y) \quad \Longleftrightarrow \quad m_{x,y}(w) \neq 0 \quad \Longleftrightarrow \quad w \in \operatorname{Supp}(x, y).$$

This proves the equivalence. □

Theorem 95 (Trichotomy of contribution classes)**.** *Fix proof trees x, y, z, w. Every left contribution class at output w belongs to one of the following three sectors:*

(i) *quotient-trivial overlap noise;*

(ii) *pure outer residual;*

(iii) *pure inner residual.*

Equivalently, the mixed residual sector is empty.

Proof. Fix a left contribution class q at output w. By the quotient-level decomposition proved earlier, q is either overlap noise or residual. If it is noise, we are done. So suppose q is residual.

Residual classes admit a second decomposition according to which kinds of left-side support they carry: pure outer, pure inner, or mixed. Thus the only possible obstruction to the announced trichotomy is the existence of a mixed residual class. We now exclude that possibility.

Assume q were mixed residual. Then q would carry both outer and inner left support while, by residuality, avoiding the quotient-trivial overlap sector. The formal geometry of two-step grafts then forces every outer representative of q into the dependent-address regime: if an outer representative had incomparable addresses, it would already lie in the commuting/transported sector and hence fail to be mixed residual. But once the addresses are comparable, the explicit witness analysis constructs right-side support from the same class, contradicting the defining characterization of the open mixed fragment as carrying no genuine right-side partner. Hence the mixed residual sector is empty.

Therefore every residual left class is either pure outer or pure inner. Adding back the overlap-noise case yields the claimed trichotomy. □

Definition 96 (Quotient sectors and transport maps)**.** *Fix proof trees x, y, z, w, and write*

$$\mathcal{L} := \mathcal{C}_{\mathrm{L}}(x, y, z; w), \qquad \mathcal{R} := \mathcal{C}_{\mathrm{R}}(x, y, z; w)$$

for the left and swapped-right contribution classes at output w. Inside these quotient sets we distinguish the following subsets:

$$\mathcal{L}_{\mathrm{out}} := \{\, q \in \mathcal{L} \mid q \textit{ has outer support} \},$$

$$\mathcal{R}_{\mathrm{out}} := \{\, q' \in \mathcal{R} \mid q' \textit{ has swapped-right outer support} \},$$

$$\mathcal{L}_{\mathrm{pin}} := \{\, q \in \mathcal{L} \mid q \textit{ is pure inner residual} \},$$

$$\mathcal{R}_{\mathrm{in,res}} := \{\, q' \in \mathcal{R} \mid q' \text{ is residual and has swapped-right inner support} \,\}.$$

The witness-level outer transport and its inverse descend to quotient maps

$$T_{\mathrm{out}} : \mathcal{L}_{\mathrm{out}} \to \mathcal{R}_{\mathrm{out}}, \qquad S_{\mathrm{out}} : \mathcal{R}_{\mathrm{out}} \to \mathcal{L}_{\mathrm{out}}.$$

Likewise the witness-level inner transport and its inverse descend to maps

$$T_{\mathrm{in}} : \mathcal{L}_{\mathrm{pin}} \to \mathcal{R}_{\mathrm{in,res}}, \qquad S_{\mathrm{in}} : \mathcal{R}_{\mathrm{in,res}} \to \mathcal{L},$$

where T_{in} is obtained by restricting the quotient-level inner transport to pure inner residual left classes, and S_{in} is the corresponding inverse transport on residual swapped-right inner classes.

Proposition 97 (Transport on quotient sectors)**.** *With notation as above:*

(i) *The outer transport maps are mutually inverse:*

$$S_{\mathrm{out}} \circ T_{\mathrm{out}} = \mathrm{id}_{\mathcal{L}_{\mathrm{out}}}, \qquad T_{\mathrm{out}} \circ S_{\mathrm{out}} = \mathrm{id}_{\mathcal{R}_{\mathrm{out}}}.$$

In particular, T_{out} is a bijection from $\mathcal{L}_{\mathrm{out}}$ onto $\mathcal{R}_{\mathrm{out}}$.

(ii) *For every $q \in \mathcal{L}_{\mathrm{pin}}$,*

$$S_{\mathrm{in}}(T_{\mathrm{in}}(q)) = q.$$

Moreover, if $t \in \mathcal{R}_{\mathrm{in,res}}$ satisfies

$$S_{\mathrm{in}}(t) = q,$$

then $t = T_{\mathrm{in}}(q)$.

Proof. For the outer sector, the formal quotient transport theorems construct T_{out} and S_{out} and prove directly that they are inverse on outer-supported classes. Concretely, if $q \in \mathcal{L}_{\mathrm{out}}$, then transport to the swapped-right outer side and transport back returns the original quotient class:

$$S_{\mathrm{out}}(T_{\mathrm{out}}(q)) = q.$$

Similarly, if $t \in \mathcal{R}_{\mathrm{out}}$, then

$$T_{\mathrm{out}}(S_{\mathrm{out}}(t)) = t.$$

These two identities are exactly the statement that T_{out} and S_{out} are mutually inverse bijections between $\mathcal{L}_{\mathrm{out}}$ and $\mathcal{R}_{\mathrm{out}}$.

For the inner sector, let $q \in \mathcal{L}_{\mathrm{pin}}$. By definition, q is pure inner residual, hence in particular inner-supported. Therefore the quotient-level inner transport is defined on q. The formal inner transport theorem shows that transporting to the swapped side and then transporting back recovers the original class:

$$S_{\mathrm{in}}(T_{\mathrm{in}}(q)) = q.$$

The same theorem also shows that $T_{\mathrm{in}}(q)$ carries swapped-right inner support, while the residuality part of the pure-inner hypothesis is preserved by transport; thus $T_{\mathrm{in}}(q) \in \mathcal{R}_{\mathrm{in,res}}$.

Now let $t \in \mathcal{R}_{\mathrm{in,res}}$ satisfy $S_{\mathrm{in}}(t) = q$. Applying T_{in} to both sides gives

$$T_{\mathrm{in}}(S_{\mathrm{in}}(t)) = T_{\mathrm{in}}(q).$$

But t is a swapped-right inner class, so the quotient-level inverse law for inner transport applies and yields $T_{\mathrm{in}}(S_{\mathrm{in}}(t)) = t$. Hence $t = T_{\mathrm{in}}(q)$, as required. □

Theorem 98 (Unique partners in the pure sectors)**.** *Fix proof trees x, y, z, w.*

(i) *For every $q \in \mathcal{L}_{\mathrm{out}}$ there exists a unique $t \in \mathcal{R}_{\mathrm{out}}$ such that*

$$S_{\mathrm{out}}(t) = q.$$

Equivalently, every outer-supported left contribution class has a unique swapped-right outer partner.

(ii) *For every $q \in \mathcal{L}_{\mathrm{pin}}$ there exists a unique $t \in \mathcal{R}_{\mathrm{in,res}}$ such that*

$$S_{\mathrm{in}}(t) = q.$$

Equivalently, every pure inner residual left contribution class has a unique residual swapped-right inner partner.

Proof. For (i), let $q \in \mathcal{L}_{\mathrm{out}}$. Set

$$t := T_{\mathrm{out}}(q) \in \mathcal{R}_{\mathrm{out}}.$$

Then by the outer inverse law,

$$S_{\mathrm{out}}(t) = S_{\mathrm{out}}(T_{\mathrm{out}}(q)) = q.$$

So a partner exists. If $t' \in \mathcal{R}_{\mathrm{out}}$ is any other class with $S_{\mathrm{out}}(t') = q$, then applying T_{out} gives

$$t' = T_{\mathrm{out}}(S_{\mathrm{out}}(t')) = T_{\mathrm{out}}(q) = t,$$

using the right-inverse identity on $\mathcal{R}_{\mathrm{out}}$. Hence the partner is unique.

For (ii), let $q \in \mathcal{L}_{\mathrm{pin}}$. Set

$$t := T_{\mathrm{in}}(q) \in \mathcal{R}_{\mathrm{in,res}}.$$

By the inner inverse law proved in the preceding proposition, one has

$$S_{\mathrm{in}}(t) = S_{\mathrm{in}}(T_{\mathrm{in}}(q)) = q.$$

So a residual swapped-right inner partner exists. If $t' \in \mathcal{R}_{\mathrm{in,res}}$ also satisfies $S_{\mathrm{in}}(t') = q$, then the uniqueness clause of the preceding proposition gives

$$t' = T_{\mathrm{in}}(q) = t.$$

Thus the residual swapped-right inner partner is unique. □

Proposition 99 (Where the quotient enters)**.** *The quotient affects the associator comparison only at the level of* two-step derivation histories*; it does not alter the* one-step *grafting support.*

Proof. The one-step statement was proved earlier: for proof trees x, y, the support of the raw bilinear product $e_x \triangleright e_y \in \mathbb{Z}[PTree]$ is exactly the set $\mathrm{Supp}(x, y)$ of one-step graft outputs. In particular, the existence of one-step composites is already determined on the raw free abelian group of proof trees.

The quotient enters only when one compares *two-step* composites. For fixed x, y, z, w, let $\mathrm{Code}(x, y, z; w)$ denote the set of raw two-step graft-pair codes with final output w, and let

$$\pi_{x,y,z;w} : \mathrm{Code}(x, y, z; w) \twoheadrightarrow \mathrm{Code}(x, y, z; w)/\sim_{\mathrm{b}}$$

be the quotient map by bureaucratic equivalence. By definition, the generators of this equivalence are the elementary commutation and transport maps introduced earlier; and by the proposition "bureaucratic equivalence preserves the composite tree", each generating move preserves the final output tree. Hence every fibre of $\pi_{x,y,z;w}$ consists of raw two-step derivation histories with the same composite output.

Therefore quotienting does not create or destroy one-step graft outputs. What it changes is the identity relation on ordered pairs of grafts occurring in the associator: schedules that differ only by bureaucratic commutation or reassociation are identified, and the overlap-noise sector becomes quotient-trivial. This is exactly the point at which the proof-theoretic bureaucracy of derivations enters the pre-Lie argument. □

10 Preparatory coproduct data and Hopf outlook

The formal development also defines a cut-based coproduct datum on proof trees. We record only the part that is presently used with mathematical confidence in the current project.

Definition 100. *An* admissible cut *of a proof tree is a finite antichain of addresses whose associated cut decomposition respects the base-interface constraints tracked in the formal development. The cut determines a pair consisting of a forest of extracted subtrees and a remainder tree.*

Proposition 101. *Every proof tree admits at least one cut decomposition.*

Proof. The empty cut is always admissible, and it yields the trivial decomposition into the empty forest together with the original tree. □

Remark 102. *This coproduct material is preparatory rather than central in the present paper. It shows that the proof-tree side already carries the decomposition data expected of a Connes–Kreimer style coproduct, but we do not yet prove the full compatibility theorems needed for a Hopf algebra in the body of the paper.*

Remark 103. *There are two natural routes from the present work to Hopf algebra.*

(i) *A direct cut/grafting route, in which one relates the cut-based coproduct to proof-tree grafting itself.*

(ii) *An Oudom–Guin route, in which the class-level pre-Lie structure proved here is first extended to a symmetric algebra and only then combined with a primitive coproduct.*

The second route is the one most strongly suggested by the present formal results, and it is the route we have in mind when the abstract mentions Hopf algebraic consequences.

Definition 104 (Stable quotient and enveloping algebra)**.** *Let $R_{\mathrm{st}} \subseteq \mathbb{Z}\langle PTree\rangle$ be the subgroup generated by the stable pre-Lie difference relations, and set*

$$Q := \mathbb{Z}\langle PTree\rangle / R_{\mathrm{st}}.$$

Write $[x] \in Q$ for the class of the basis element e_x. The class-level pre-Lie product on Q induces a Lie bracket

$$[u, v] := u \triangleright v - v \triangleright u,$$

and we denote the resulting Lie algebra by Q_{Lie}. Its universal enveloping algebra is denoted

$$U(Q) := U(Q_{\mathrm{Lie}}).$$

Finally, write

$$\iota : Q \to U(Q)$$

for the canonical linear map.

Theorem 105 (Conditional descent of primitive generator data)**.** *Let Q be the stable quotient of the free abelian group on proof trees by the pre-Lie difference stable relations, let $U(Q)$ be the universal enveloping algebra of the induced Lie algebra, and let $\iota : Q \to U(Q)$ denote the canonical map. Suppose that one is given generator-level data*

$$\Delta_0 : PTree \to U(Q) \otimes U(Q), \qquad \varepsilon_0 : PTree \to \mathbb{Z}$$

whose induced linear maps on $\mathbb{Z}[PTree]$ vanish on the stable relations. Then there exist well-defined descended linear maps

$$\Delta : Q \to U(Q) \otimes U(Q), \qquad \varepsilon : Q \to \mathbb{Z}$$

whose values on tree generators are exactly those prescribed by Δ_0 and ε_0. In particular, if

$$\Delta_0(x) = \iota([x]) \otimes 1 + 1 \otimes \iota([x]), \qquad \varepsilon_0(x) = 0$$

on generators, then the descended maps satisfy the same formulas on generator classes:

$$\Delta([x]) = \iota([x]) \otimes 1 + 1 \otimes \iota([x]), \qquad \varepsilon([x]) = 0.$$

Proof. Let

$$\pi : \mathbb{Z}[PTree] \twoheadrightarrow Q$$

be the quotient map. The hypotheses say precisely that the linear extensions of Δ_0 and ε_0 to $\mathbb{Z}[PTree]$ vanish on the stable relation submodule. Therefore each of them factors uniquely through the quotient Q. In other words, there exist unique linear maps

$$\Delta : Q \to U(Q) \otimes U(Q), \qquad \varepsilon : Q \to \mathbb{Z}$$

such that

$$\Delta \circ \pi = \widetilde{\Delta}_0, \qquad \varepsilon \circ \pi = \widetilde{\varepsilon}_0,$$

where $\widetilde{\Delta}_0$ and $\widetilde{\varepsilon}_0$ denote the linear extensions of the generator data.

Evaluating these identities on a basis tree x gives

$$\Delta([x]) = \Delta(\pi(e_x)) = \widetilde{\Delta}_0(e_x) = \Delta_0(x), \qquad \varepsilon([x]) = \varepsilon(\pi(e_x)) = \widetilde{\varepsilon}_0(e_x) = \varepsilon_0(x).$$

Hence the descended maps agree with the prescribed generator formulas. The primitive-generator case is obtained by substituting $\Delta_0(x) = \iota([x]) \otimes 1 + 1 \otimes \iota([x])$ and $\varepsilon_0(x) = 0$. □

Theorem 106 (Extension principle from generators)**.** *Let $V = \mathbb{Z}[PTree]$. Any bilinear product on V whose values on tree generators satisfy the right pre-Lie identity extends to a right pre-Lie algebra structure on all of V. Likewise, any additive weight law verified on generator products extends to all homogeneous elements of V.*

Proof. Write

$$A(a, b, c) := (a \cdot b) \cdot c - a \cdot (b \cdot c)$$

for the associator of the bilinear product. The difference

$$T(a, b, c) := A(a, b, c) - A(a, c, b)$$

is trilinear in a, b, c, because the product is bilinear. If T vanishes on all triples of basis elements e_x, e_y, e_z, then by trilinearity it vanishes on all finite $\mathbb{Z}$-linear combinations of such basis elements. Since $V = \mathbb{Z}[PTree]$ is the free abelian group on the trees, every element of V is a finite sum of basis vectors, so $T = 0$ on all of V^3. This is exactly the right pre-Lie identity on V.

The grading statement is analogous. Let $V_n \subseteq V$ be the subgroup generated by trees of weight n. If the generator-level law says that the product of a weight-m basis tree and a weight-n basis tree lies in V_{m+n}, then bilinearity implies that the product of arbitrary homogeneous elements of degrees m and n is a finite sum of terms each lying in V_{m+n}. Hence the whole product lies in V_{m+n}, so the weight law extends from generators to all homogeneous elements. □

Remark 107 (Current state of the Oudom–Guin development)**.** *The formal development already contains preparatory work toward route (ii). The file* `GrossmanLarssonOudomGuin.lean` *establishes, without any* `sorry`*, the following:*

- *a graft-compatible weight function on proof trees, grading every one-step product output by* $\mathsf{weight}(w) = \mathsf{weight}(x) + \mathsf{weight}(y)$ *whenever* $w \in \mathit{Graft}(x, y)$*;*
- *a proof that any bilinear product on the linear proof-tree carrier whose generator-level behaviour matches the quotient pre-Lie law automatically satisfies the right pre-Lie axiom on the entire carrier (`generatorRightPreLieLaw_extends`);*
- *a proof that any such product respects the weight grading on all homogeneous elements (`generatorWeightLaw_extends`).*

The remaining step for route (ii) is to construct the specific bilinear product from the quotient data — i.e. to show that the grafting operation descends to a well-defined map on the linear carrier that is compatible with the quotient — and then to apply the Oudom–Guin theorem to obtain the enveloping Hopf algebra on the symmetric algebra. The infrastructure is in place; this is the content of the next paper in this series.

Remark 108 (Why the Connes–Kreimer route is not taken)**.** *The direct cut/grafting route (i) would require the algebraic product to be the forest product (disjoint union of proof trees). However, a forest of proof trees has no natural proof-theoretic interpretation: there is no sequent whose derivation* is *a side-by-side collection of independent proof trees. Proof composition is sequential — one proof's conclusion becomes another's premiss leaf — not parallel. The grafting product is proof-theoretically meaningful precisely because it is proof composition. Route (i) is therefore not pursued here, though a comparison with Connes–Kreimer-style Hopf algebras by forgetting the sequent decoration remains an interesting open problem (see the following remark).*

Remark 109. *A further comparison problem, not treated here, is to relate the present structure to familiar rooted-tree Hopf algebras by forgetting some of the proof-theoretic decoration while retaining the underlying tree and insertion geometry. Such comparison theorems would strengthen the claim that the present construction refines, rather than competes with, established combinatorial Hopf-algebraic frameworks. The key question is whether the Hopf algebra obtained via the Oudom–Guin route depends essentially on the primitive relation* $\mid\!\sim_0$ *— in the sense that different choices of* $\mid\!\sim_0$ *yield genuinely non-isomorphic algebraic structures — or whether the structure is universal and the logical content enters only through the choice of basis. Settling this would determine whether the construction encodes something specifically about nonmonotonic proof theory or merely recovers a combinatorial object by a proof-theoretic route.*

Remark 110 (Structural rules as future parameters)**.** *As noted in the introduction, the present paper fixes multiset contexts and omits weakening and contraction. Each structural rule is a potential parameter. Adding weakening makes sequent interfaces non-rigid: one could graft into any leaf by first weakening the context to match. This would change the leaf-matching constraint fundamentally and is*

expected to obstruct the coproduct compatibility rather than the pre-Lie structure. Adding contraction collapses multisets differing in formula multiplicities, changing which sequents are identified and potentially breaking the noise/residual separation whose geometry depends on the counting of matching leaves. Moving from multisets to sequences (removing the built-in exchange) would add ordered structure to sequents and make the grafting product sensitive to context ordering in ways that require separate analysis. A systematic study of which structural rules are obstructions to Hopf-algebraic compatibility is left for future work.

11 Conclusion

We have constructed a proof-theoretic route from a primitive consequence relation to a class-level pre-Lie identity on proof-tree composition. Formally, the route is

$$\begin{aligned} \mid\!\sim_0 &\longrightarrow NMMS(\mid\!\sim_0) \\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting geometry} \\ &\longrightarrow \text{graft-pair quotient} \longrightarrow \text{class-level pre-Lie identity.} \end{aligned}$$

The distinctive point is not merely that rooted trees appear, but that they appear as proof objects carrying exact sequent interfaces. Grafting is therefore proof composition, not arbitrary combinatorial insertion. The graft-pair quotient then isolates what should count as invariant in proof composition: independent commutations and quotient-trivial overlap disappear as bureaucracy, while the residual sector becomes pure and rigid enough to satisfy the class-level pre-Lie identity.

From the standpoint of logic, this means that a nonmonotonic proof theory can carry nontrivial algebraic structure without first collapsing to a monotone or structurally saturated setting. From the standpoint of algebra, it shows that pre-Lie behavior can emerge from proof composition once identity of proof is handled carefully at the level of graft-pair interactions. The Hopf-theoretic extensions suggested by the present development are important, but they are best seen as a next step built on top of the logic result already proved here: after quotienting bureaucratic symmetries, proof composition on these nonmonotonic proof trees satisfies a class-level pre-Lie identity.

The quotient is not a stylistic choice. The direct approach to the pre-Lie identity — proving that the raw witness counts balance on both sides of the associator — was attempted and did not close. The equivalence relation on graft-pair codes, and in particular the `outer_comm_inner` identification of comparable-address graftings, is the structure that makes the proof work. This identification has a proof-theoretic reading: it says that two derivation histories for the same output proof tree, which look geometrically different at the address level, should count as the same composition when the relevant graft sites stand in a dependency relation. The noise/residual separation is then a precise account of which histories are genuinely the same (noise, canceling from the associator) and which are genuinely distinct (residual, matched uniquely across the two bracketings).

The central result is fully machine-verified in Lean 4. The class-level pre-Lie identity depends only on the standard axioms of Lean's foundational system. The broader programme — determining which structural rules obstruct Hopf-algebraic compatibility, and whether the resulting structure encodes specific properties of the primitive consequence relation $\mid\!\sim_0$ — remains open and will be pursued in subsequent work.